PES UNIVERSITY

Department of Computer Science and Engineering

DBMS Experiential Learning – Level 3 Project

PROJECT REPORT

Corporate Recruitment & Candidate Resume Indexing System

RecruitIQ – MySQL + Qdrant vector database + Next.js

Item	Details
Team	Team 10
Team members	Srujan Yalla (PES2UG24CS527) , Sumedh B Rao (PES2UG24CS535)
Databases	MySQL 8 (relational, source of truth) + Qdrant (vector database)
Application stack	Next.js 14 (App Router + Route Handlers), JWT authentication, role-based access
Academic period	October 2026

Contents

1. Objectives and Functional Requirements
2. ER Diagram and Relational Schema
3. Database Choice and Justification
4. Data Dictionary
5. SQL and Vector Database Implementation
6. Important Queries
7. System Architecture
8. Application Screenshots
9. Testing Results
10. Conclusion

1. Objectives and Functional Requirements

1.1 Problem statement

Companies receive hundreds of applications per opening, and recruiters struggle to find the right candidates in free-text resumes. Candidates, in turn, cannot easily tell which openings suit their skills. This project builds a recruitment platform that manages the full hiring lifecycle and goes beyond CRUD: business rules are enforced inside the database, hiring is transactional, jobs are scored against candidate skills, and resumes and job descriptions are matched by meaning using vector embeddings.

1.2 Objectives

- Model the hiring lifecycle (company, department, job, application, interview, hire) in a normalised MySQL schema with integrity enforced at the database level.
- Demonstrate advanced SQL: views, stored function and procedures, triggers, transactions with row locking, indexes and full-text search.
- Add semantic search with a vector database so that resumes and job descriptions can be matched on meaning rather than keywords.
- Provide a secure, role-based Next.js web application (Admin, Recruiter, Candidate) with no credentials hard-coded in the source.
- Keep the code in Git/GitHub with documented setup and test cases.

1.3 Functional requirements

ID	Requirement	Role
FR1	Register and log in with hashed passwords; JWT session in an HttpOnly cookie	All
FR2	Maintain profile and skills; upload multiple resumes	Candidate
FR3	Search jobs (full-text, location, type, salary) and see skill-match percentage	Candidate
FR4	Apply to a job (only open jobs, resume mandatory, no duplicate applications)	Candidate
FR5	Post, close and reopen jobs with required skills	Recruiter
FR6	Review applications, change status; every change is audit-logged	Recruiter
FR7	Schedule, complete and cancel interviews	Recruiter
FR8	Hire a candidate atomically (reject other applicants, close the job)	Recruiter
FR9	Analytics dashboard: funnel, salary by industry, top skills	Recruiter, Admin
FR10	Semantic search: recommended jobs for a resume; matching resumes for a job	Candidate, Recruiter
FR11	Global visibility of all data; job deletion	Admin

1.4 Coverage of the Level 3 requirements

Requirement	Where it is satisfied
SQL + vector database (Option 2)	MySQL 8 for relational data; Qdrant for resume and job embeddings (Sections 3, 5)
ER diagram and relational schema	Section 2 (updated to match the implementation)
DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	Section 5.1 to 5.3
DML: INSERT, UPDATE, DELETE, SELECT and complex queries	Sections 5.5 and 6
At least four advanced SQL concepts + one vector concept	Transactions, views, function and procedures, triggers, indexing, full-text search; vector similarity search (Section 5.7)
Next.js web interface and API layer	Section 8; 13 route handlers, 8 pages
Credentials not hard-coded	Environment variables in .env (Section 7)
Authentication and at least two roles	bcrypt + JWT; Admin, Recruiter, Candidate (Section 7.3)
Data validation and integrity at database level	NOT NULL, UNIQUE, CHECK, FK, triggers; tested in Section 9
Testing	23 test cases: constraints, triggers, procedures, security and semantic search (Section 9)

2. ER Diagram and Relational Schema

2.1 Initial design (hand-drawn)

The first design was drawn by hand during the planning phase. It identified the core entities (Company, Department, Job, Candidate, Resume, Skill, Application, Interview) and their relationships.

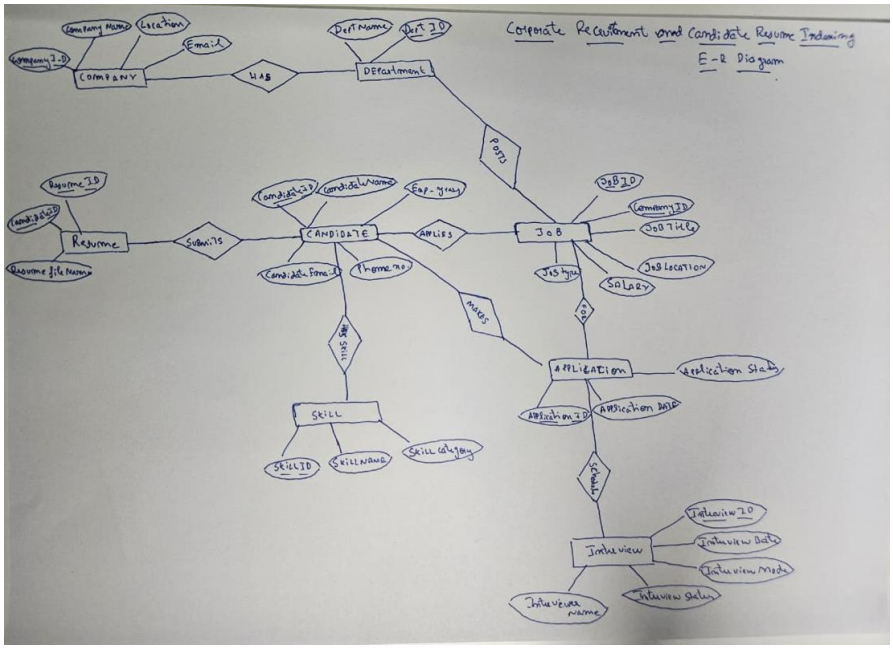


Figure 1: Initial hand-drawn ER diagram (design draft)

2.2 Changes made during implementation

While implementing the database, the design was refined. The table below lists every difference between the initial draft and the final, executed schema; the updated ER diagram in Section 2.3 reflects all of them.

Initial draft	Final implementation	Reason
Recruiter entity	Replaced by the Users entity (Role = Recruiter, linked to Company)	One authentication table serves all roles (RBAC)
Candidate_Phone, Education, Experience entities	Phone, Qualification and Experience_Years as Candidate attributes	Simpler 3NF design; matches the executed DDL
Job_Posting (Dept_ID, Recruiter_ID)	Job with Company_ID, optional Dept_ID, Posted_Date and Status	Open/closed lifecycle; company-level ownership
Skill relates to Candidate only	New Job_Skill (M:N between Job and Skill)	Enables the skill-match function
No authentication entity	New Users entity	Login with hashed passwords and roles
No history of status changes	New Application_Status_Log (filled by trigger)	Audit trail for every application
Interview (as drawn)	Interview with Interviewer_Name, Interview_Date, Mode, Status and Feedback	Matches the drawn ER; adds integrity constraints

2.3 Updated ER diagram

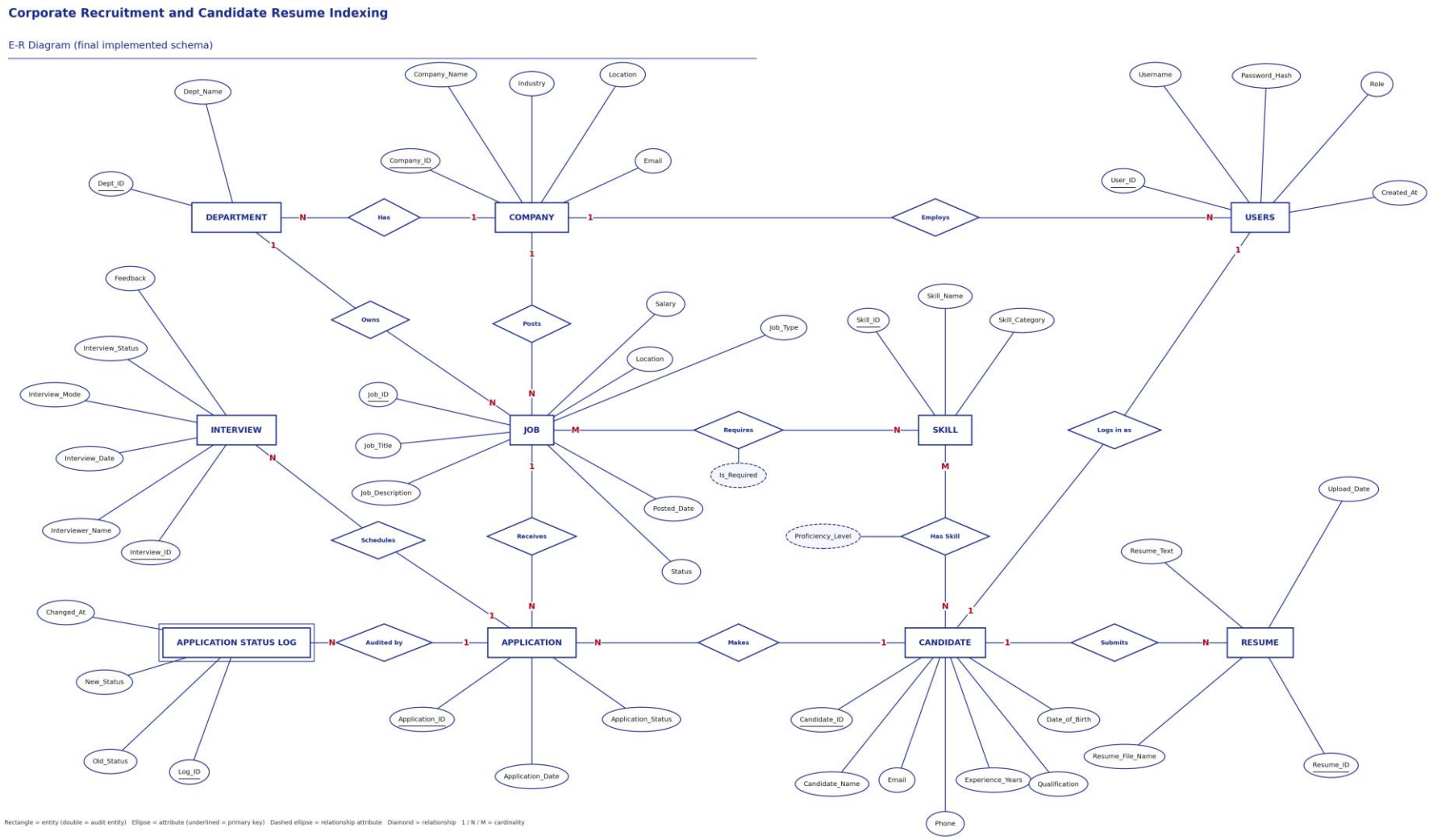


Figure 2: Updated ER diagram of the implemented database (10 entities, 12 relationships)

2.4 Relational schema

The ER diagram was converted to the relational schema below. Many-to-many relationships (Job–Skill, Candidate–Skill) are resolved into junction tables with composite primary keys. Primary keys are bold and underlined, foreign keys are in italics, and arrows point from a foreign key to the table it references.

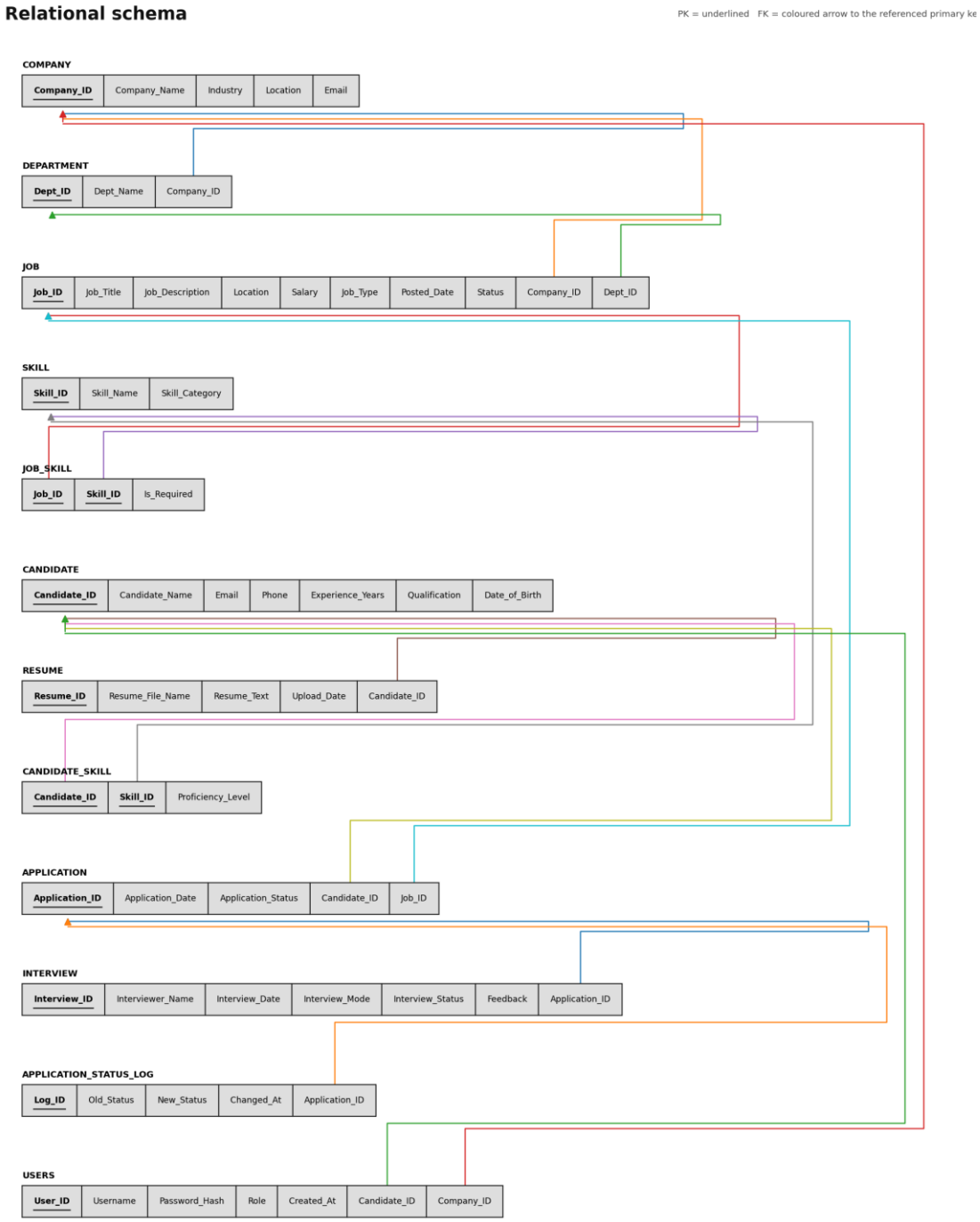


Figure 3: Relational schema with primary keys, foreign keys and references

Schema in relational notation

```
Company(Company_ID PK, Company_Name, Industry, Location, Email UQ)
Department(Dept_ID PK, Company_ID FK, Dept_Name)
    UQ(Company_ID, Dept_Name)
Job(Job_ID PK, Company_ID FK, Dept_ID FK null, Job_Title, Job_Description, Location,
    Salary, Job_Type, Posted_Date, Status)
Skill(Skill_ID PK, Skill_Name UQ, Skill_Category)
Job_Skill(Job_ID FK, Skill_ID FK, Is_Required)
    PK(Job_ID, Skill_ID)
Candidate(Candidate_ID PK, Candidate_Name, Email UQ, Phone UQ, Experience_Years,
          Qualification, Date_of_Birth)
Candidate_Skill(Candidate_ID FK, Skill_ID FK, Proficiency_Level)
    PK(Candidate_ID, Skill_ID)
Resume(Resume_ID PK, Candidate_ID FK, Resume_File_Name, Resume_Text, Upload_Date)
Application(Application_ID PK, Candidate_ID FK, Job_ID FK, Application_Date,
            Application_Status)
    UQ(Candidate_ID, Job_ID)
Interview(Interview_ID PK, Application_ID FK, Interviewer_Name, Interview_Date,
          Interview_Mode, Interview_Status, Feedback)
Application_Status_Log(Log_ID PK, Application_ID FK, Old_Status, New_Status,
    Changed_At)
Users(User_ID PK, Username UQ, Password_Hash, Role,
      Candidate_ID FK null, Company_ID FK null, Created_At)
```

Referential actions

Foreign key	ON DELETE	Reasoning
Job.Company_ID → Company	RESTRICT	A company with jobs cannot be deleted
Job.Dept_ID → Department	SET NULL	Removing a department keeps its jobs
Department.Company_ID → Company	CASCADE	Departments belong to a company
Resume.Candidate_ID → Candidate	CASCADE	Resumes have no meaning without the candidate
Candidate_Skill → Candidate / Skill	CASCADE	Junction rows removed with either parent
Job_Skill → Job / Skill	CASCADE	Junction rows removed with either parent
Application → Candidate / Job	RESTRICT	Hiring history must never be lost silently
Interview.Application_ID → Application	CASCADE	Interviews belong to an application
Users.Candidate_ID → Candidate	CASCADE	Login removed with the candidate
Users.Company_ID → Company	SET NULL	Recruiter account survives company removal

Normalisation

All tables are in third normal form (and BCNF): every non-key attribute depends on the key, the whole key and nothing but the key. Company data is stored once and referenced from Job; skill names are stored once in Skill and referenced from Candidate_Skill and Job_Skill; multi-valued facts (skills, resumes, interviews) live in their own tables.

3. Database Choice and Justification

The project uses two database paradigms (Option 2: SQL + Vector database). Each type of data is stored where its nature fits best.

Data	Store	Why
Companies, departments, jobs, candidates, applications, interviews, users, skills	**MySQL 8**	Strongly structured and highly relational. Needs ACID transactions (hiring), foreign keys, CHECK/UNIQUE constraints, triggers, views and aggregate queries with joins.
Resume text and job descriptions (unstructured, text-heavy)	**Qdrant (vector database)**	Keyword search cannot match "ML engineer" with "builds recommendation models". Sentence embeddings with cosine similarity retrieve semantically similar resumes and jobs.

Why not a document database such as MongoDB?

The data is relational (a hire touches Application, Job and the audit log atomically) and needs enforced integrity. The only unstructured part needs similarity search rather than flexible documents, which a vector database provides directly.

How the two stores work together

MySQL remains the single source of truth. Qdrant holds a derived index of resume and job vectors, keyed by Resume_ID and Job_ID. A semantic query is answered in two steps: Qdrant returns the nearest IDs, then MySQL filters them (for example only open jobs) and enriches them with names, skills and the skill-match percentage. If the vector index is lost it can be rebuilt from MySQL with one command.

Why Qdrant and a local embedding model

- Qdrant is a purpose-built vector database with HNSW indexing, runs in one Docker container and has a web dashboard.
- The all-MiniLM-L6-v2 model (384 dimensions) runs locally through ONNX, so no API key or network service is required and no resume text leaves the machine.

4. Data Dictionary

Constraint names in brackets are the names used in the DDL and shown in the error messages of Section 9.

Company

Column	Data type	Constraints	Description
Company_ID	INT	PK, AUTO_INCREMENT	Unique company identifier
Company_Name	VARCHAR(100)	NOT NULL	Registered name
Industry	VARCHAR(50)		Industry sector
Location	VARCHAR(100)		Head-office city
Email	VARCHAR(100)	UNIQUE	HR contact email

Department

Column	Data type	Constraints	Description
Dept_ID	INT	PK, AUTO_INCREMENT	Department identifier
Company_ID	INT	NOT NULL, FK → Company, CASCADE	Owning company
Dept_Name	VARCHAR(80)	NOT NULL, UNIQUE(Company_ID, Dept_Name)	Department name

Job

Column	Data type	Constraints	Description
Job_ID	INT	PK, AUTO_INCREMENT	Job identifier
Company_ID	INT	NOT NULL, FK → Company	Hiring company
Dept_ID	INT	FK → Department, SET NULL	Optional department
Job_Title	VARCHAR(100)	NOT NULL	Title of the opening
Job_Description	TEXT	FULLTEXT (ft_job)	Duties; embedded into Qdrant
Location	VARCHAR(100)	Index idx_job_status_location	Work location
Salary	DECIMAL(10,2)	CHECK ≥ 0 (CHK_Salary)	Annual salary
Job_Type	VARCHAR(30)	CHECK IN Full-time, Part-time, Contract, Internship (CHK_Job_Type)	Employment type
Posted_Date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Posting date
Status	VARCHAR(10)	NOT NULL, DEFAULT Open, CHECK IN Open, Closed (CHK_Job_Status)	Opening lifecycle state

Skill and Job_Skill

Column	Data type	Constraints	Description
Skill.Skill_ID	INT	PK, AUTO_INCREMENT	Skill identifier

Column	Data type	Constraints	Description
Skill.Skill_Name	VARCHAR(50)	NOT NULL, UNIQUE	Skill name, stored once
Skill.Skill_Category	VARCHAR(50)		Programming, Database, AI, …
Job_Skill.Job_ID	INT	PK, FK → Job, CASCADE	Job requiring the skill
Job_Skill.Skill_ID	INT	PK, FK → Skill, CASCADE	Required skill
Job_Skill.Is_Required	BOOLEAN	NOT NULL, DEFAULT TRUE	Mandatory or preferred

Candidate

Column	Data type	Constraints	Description
Candidate_ID	INT	PK, AUTO_INCREMENT	Candidate identifier
Candidate_Name	VARCHAR(100)	NOT NULL	Full name
Email	VARCHAR(100)	NOT NULL, UNIQUE	Contact email
Phone	VARCHAR(15)	UNIQUE, CHECK length ≥ 10 (CHK_Phone)	Contact number
Experience_Years	DECIMAL(4,1)	DEFAULT 0, CHECK ≥ 0 (CHK_Experience)	Total experience
Qualification	VARCHAR(100)		Highest qualification
Date_of_Birth	DATE	Added with ALTER TABLE	Date of birth

Resume and Candidate_Skill

Column	Data type	Constraints	Description
Resume.Resume_ID	INT	PK, AUTO_INCREMENT	Resume identifier
Resume.Candidate_ID	INT	NOT NULL, FK → Candidate, CASCADE	Owner
Resume.Resume_File_Name	VARCHAR(150)	NOT NULL	Uploaded file name
Resume.Resume_Text	TEXT	FULLTEXT (ft_resume)	Extracted text; embedded into Qdrant
Resume.Upload_Date	DATE	NOT NULL	Upload date
Candidate_Skill.Candidate_ID	INT	PK, FK → Candidate, CASCADE	Candidate
Candidate_Skill.Skill_ID	INT	PK, FK → Skill, CASCADE	Skill known
Candidate_Skill.Proficiency_Level	VARCHAR(30)	CHECK IN Beginner, Intermediate, Advanced, Expert	Skill level

Application

Column	Data type	Constraints	Description
Application_ID	INT	PK, AUTO_INCREMENT	Application identifier
Candidate_ID	INT	NOT NULL, FK → Candidate	Applicant

Column	Data type	Constraints	Description
Job_ID	INT	NOT NULL, FK → Job, UNIQUE(Candidate_ID, Job_ID)	Applied job; one application per job
Application_Date	DATE	NOT NULL	Date applied
Application_Status	VARCHAR(30)	DEFAULT Applied, CHECK IN Applied, Shortlisted, Interview Scheduled, Rejected, Hired (CHK_App_Status)	Pipeline state

Interview

Column	Data type	Constraints	Description
Interview_ID	INT	PK, AUTO_INCREMENT	Interview identifier
Application_ID	INT	NOT NULL, FK → Application, CASCADE	Related application
Interviewer_Name	VARCHAR(100)	NOT NULL	Panel member
Interview_Date	DATETIME	NOT NULL, index idx_interview_date	Scheduled time
Interview_Mode	VARCHAR(10)	CHECK IN Online, Onsite, Phone	Mode
Interview_Status	VARCHAR(10)	CHECK IN Scheduled, Completed, Cancelled	State
Feedback	TEXT		Feedback after the interview

Users and Application_Status_Log

Column	Data type	Constraints	Description
Users.User_ID	INT	PK, AUTO_INCREMENT	Account identifier
Users.Username	VARCHAR(50)	NOT NULL, UNIQUE	Login name
Users.Password_Hash	VARCHAR(100)	NOT NULL	bcrypt hash, never plain text
Users.Role	VARCHAR(15)	CHECK IN Admin, Recruiter, Candidate	Access role
Users.Candidate_ID	INT	FK → Candidate, UNIQUE, CASCADE	Set for the Candidate role
Users.Company_ID	INT	FK → Company, SET NULL	Set for the Recruiter role
Log.Log_ID	INT	PK, AUTO_INCREMENT	Log entry
Log.Application_ID	INT	NOT NULL, FK → Application, CASCADE	Audited application
Log.Old_Status / New_Status	VARCHAR(30)	New_Status NOT NULL	Status before and after (written by trigger)
Log.Changed_At	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	When the change happened

5. SQL and Vector Database Implementation

The database was built in stages using MySQL Workbench. Each script was executed against the CorporateRecruitmentDB database and the output captured below.

Script	Purpose	Demonstrates
01	Base DDL: 7 tables, ALTER and DROP demonstrations	CREATE, ALTER, DROP, PK/FK, constraints
02	Schema extensions and indexes	ALTER, new tables, CHECK constraints, indexes, FULLTEXT
03	Views	Views over joins and aggregates
04	Function, procedures, triggers	Functions, stored procedures, transactions, triggers
05	Seed data	INSERT, UPDATE
06	Important queries	SELECT, JOIN, GROUP BY, HAVING, subqueries, DML
07	Test cases	Constraint and trigger validation

5.1 Base DDL (script 01)

The seven base tables (Company, Candidate, Job, Resume, Skill, Candidate_Skill, Application) were created first, with primary keys, foreign keys, UNIQUE, NOT NULL, DEFAULT and CHECK constraints (screenshots are part of the separate DDL submission). The ALTER and DROP statements of this script:

```
ALTER TABLE Candidate ADD Date_of_Birth DATE;

ALTER TABLE Candidate
  ADD CONSTRAINT CHK_Phone CHECK (Phone IS NULL OR CHAR_LENGTH(Phone) >= 10);

CREATE TABLE Temporary_Data (Temp_ID INT, Temp_Name VARCHAR(50));
DROP TABLE Temporary_Data;
```

5.2 Schema extensions and indexes (script 02)

The schema was extended with ALTER TABLE and CREATE TABLE statements so that it matches the final design: Department, Job_Skill, Interview, Users and Application_Status_Log were created; Job received Dept_ID, Posted_Date and Status; CHECK constraints were added to Job, Application and Candidate_Skill.

```
ALTER TABLE Job
  ADD COLUMN Dept_ID INT NULL,
  ADD COLUMN Posted_Date DATE NOT NULL DEFAULT (CURRENT_DATE),
  ADD COLUMN Status VARCHAR(10) NOT NULL DEFAULT 'Open',
  ADD CONSTRAINT FK_Job_Dept FOREIGN KEY (Dept_ID)
      REFERENCES Department(Dept_ID) ON DELETE SET NULL,
  ADD CONSTRAINT CHK_Job_Status CHECK (Status IN ('Open','Closed')),
  ADD CONSTRAINT CHK_Job_Type CHECK (Job_Type IS NULL OR
      Job_Type IN ('Full-time','Part-time','Contract','Internship'));

CREATE INDEX idx_job_status_location ON Job(Status, Location);
CREATE INDEX idx_app_job_status      ON Application(Job_ID, Application_Status);
CREATE INDEX idx_candidate_exp       ON Candidate(Experience_Years);
CREATE INDEX idx_resume_cand_date    ON Resume(Candidate_ID, Upload_Date);
CREATE INDEX idx_interview_date      ON Interview(Interview_Date);
CREATE FULLTEXT INDEX ft_job    ON Job(Job_Title, Job_Description);
```

```
CREATE FULLTEXT INDEX ft_resume ON Resume(Resume_Text);
CREATE INDEX idx_tmp ON Company(Industry);   DROP INDEX idx_tmp ON Company;
```

#	Time	Action	Message	Duration / Fetch
1	22:55:45	USE CorporateRecruitmentDB	0 row(s) affected	0.016 sec
2	22:55:45	CREATE TABLE Department (Dept_ID INT AUTO_INCREMENT, Company_ID IN...	0 row(s) affected	0.203 sec
3	22:55:45	ALTER TABLE Job ADD COLUMN Dept_ID INT NULL, ADD COLUMN Posted_...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
4	22:55:46	CREATE TABLE Job_Skill (Job_ID INT, Skill_ID INT, Is_Required BOOLEA...	0 row(s) affected	0.031 sec
5	22:55:46	ALTER TABLE Application ADD CONSTRAINT CHK_App_Status CHECK (Application...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
6	22:55:46	ALTER TABLE Candidate_Skill ADD CONSTRAINT CHK_Proficiency CHECK (Proficie...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.063 sec
7	22:55:46	CREATE TABLE Interview (Interview_ID INT AUTO_INCREMENT, Application_I...	0 row(s) affected	0.031 sec
8	22:55:46	CREATE TABLE Users (User_ID INT AUTO_INCREMENT, Username VARC...	0 row(s) affected	0.031 sec
9	22:55:46	CREATE TABLE Application_Status_Log (Log_ID INT AUTO_INCREMENT PRI...	0 row(s) affected	0.031 sec
10	22:55:46	CREATE INDEX idx_job_status_location ON Job(Status, Location)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.032 sec
11	22:55:46	CREATE INDEX idx_app_job_status ON Application(Job_ID, Application_Status)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
12	22:55:46	CREATE INDEX idx_candidate_exp ON Candidate(Experience_Years)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
13	22:55:46	CREATE INDEX idx_resume_cand_date ON Resume(Candidate_ID, Upload_Date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
14	22:55:46	CREATE INDEX idx_interview_date ON Interview(Interview_Date)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
15	22:55:46	CREATE FULLTEXT INDEX ft_job ON Job(Job_Title, Job_Description)	0 row(s) affected, 1 warning(s): 124 InnoDB rebuilding table to add column FTS_DOC_ID ...	0.172 sec
16	22:55:46	CREATE FULLTEXT INDEX ft_resume ON Resume(Resume_Text)	0 row(s) affected, 1 warning(s): 124 InnoDB rebuilding table to add column FTS_DOC_ID ...	0.141 sec
17	22:55:46	CREATE INDEX idx_tmp ON Company(Industry)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.031 sec
18	22:55:46	DROP INDEX idx_tmp ON Company	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.015 sec
19	22:55:46	SHOW INDEX FROM Job	7 row(s) returned	0.016 sec / 0.000 sec

Figure 4: Action output of script 02: all 19 statements executed (the two warnings are InnoDB adding the hidden FTS_DOC_ID column for the first FULLTEXT index)

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment
job	0	PRIMARY	1	Job_ID	A	0	NULL	NULL		BTREE	
job	1	FK_Job_Company	1	Company_ID	A	0	NULL	NULL		BTREE	
job	1	FK_Job_Dept	1	Dept_ID	A	0	NULL	NULL	YES	BTREE	
job	1	idx_job_status_location	1	Status	A	0	NULL	NULL		BTREE	
job	1	idx_job_status_location	2	Location	A	0	NULL	NULL	YES	BTREE	
job	1	ft_job	1	Job_Title	NULL	0	NULL	NULL		FULLTEXT	
job	1	ft_job	2	Job_Description	NULL	0	NULL	NULL	YES	FULLTEXT	

Result 1 | Read Only | Context Help | Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	22:55:45	USE CorporateRecruitmentDB	0 row(s) affected	0.016 sec
2	22:55:45	CREATE TABLE Department (Dept_ID INT AUTO_INCREMENT, Company_ID ...	0 row(s) affected	0.203 sec

Figure 5: SHOW INDEX FROM Job: primary key, foreign keys, composite index and FULLTEXT index ft_job

Tables_in_corporaterecruitmentdb
application
application_status_log
candidate
candidate_skill
company
department
interview
job
job_skill
resume
skill
users

Figure 6: SHOW TABLES: the 12 tables of the final schema

5.3 Views (script 03)

Three views encapsulate frequently used joins and aggregates. They are used directly by the application (dashboard, candidate search and interview schedule).

View	Content
vw_job_summary	Job with company, department, number of applications and number of hires (JOIN, LEFT JOIN, GROUP BY, COUNT, SUM)
vw_candidate_profile	Candidate with all skills concatenated using GROUP_CONCAT and the number of applications (correlated subquery)
vw_interview_schedule	Interview joined with application, candidate, job and company

```
CREATE OR REPLACE VIEW vw_job_summary AS
SELECT j.Job_ID, j.Job_Title, c.Company_ID, c.Company_Name, d.Dept_Name,
       j.Location, j.Salary, j.Status,
       COUNT(a.Application_ID) AS Total_Applications,
       COALESCE(SUM(a.Application_Status='Hired'),0) AS Hired_Count
FROM Job j
JOIN Company c          ON c.Company_ID = j.Company_ID
LEFT JOIN Department d  ON d.Dept_ID    = j.Dept_ID
LEFT JOIN Application a ON a.Job_ID     = j.Job_ID
GROUP BY j.Job_ID, j.Job_Title, c.Company_ID, c.Company_Name, d.Dept_Name,
         j.Location, j.Salary, j.Status;
```

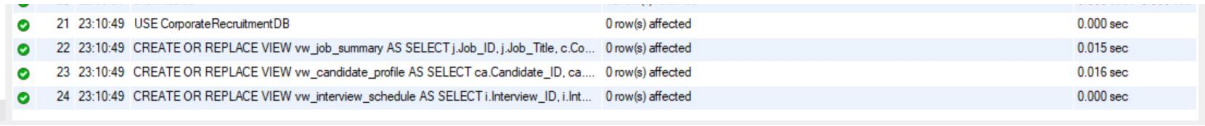


Figure 7: Views created successfully

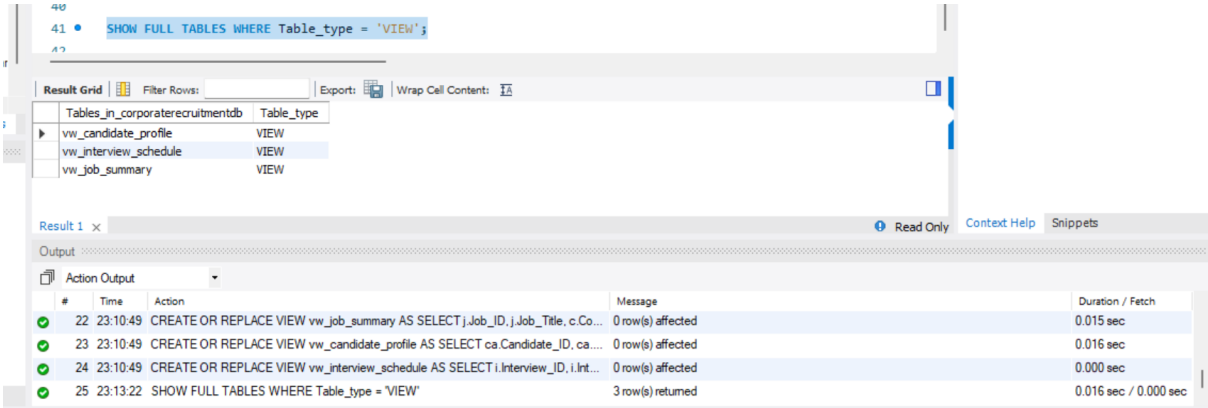


Figure 8: SHOW FULL TABLES WHERE Table_type = VIEW: the three views

5.4 Function, stored procedures and triggers (script 04)

Object	Purpose
fn_match_percentage(candidate, job)	Returns the percentage of a job's required skills that the candidate has
sp_apply_for_job(candidate, job)	Inserts an application inside a transaction; rolls back on any error
sp_hire_candidate(application)	Locks the row (FOR UPDATE), marks it Hired, rejects all other open applicants and closes the job as one atomic transaction
trg_app_before_insert	Blocks applications to closed jobs and applications without a resume
trg_app_before_update	Makes "Hired" a final state
trg_app_after_insert / after_update	Writes every status change to Application_Status_Log (audit trail)
trg_interview_before_insert	Rejects an interview dated before the application date

Object	Purpose
trg_interview_after_insert	Moves the application to “Interview Scheduled” automatically

The hiring procedure shows how a transaction keeps data consistent. If any statement fails, the EXIT HANDLER rolls back everything, so a job can never be closed without a hired candidate or vice versa.

```
CREATE PROCEDURE sp_hire_candidate(IN p_app INT)
BEGIN
  DECLARE v_job INT DEFAULT NULL;  DECLARE v_status VARCHAR(30);
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
    SELECT Job_ID, Application_Status INTO v_job, v_status
    FROM Application WHERE Application_ID = p_app FOR UPDATE;
    IF v_job IS NULL THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Application not
        found'; END IF;
    IF v_status NOT IN ('Shortlisted','Interview Scheduled') THEN
      SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT=
        'Only shortlisted / interviewed applications can be hired'; END IF;
    UPDATE Application SET Application_Status='Hired' WHERE Application_ID = p_app;
    UPDATE Application SET Application_Status='Rejected'
     WHERE Job_ID=v_job AND Application_ID<>p_app AND Application_Status NOT IN
         ('Hired','Rejected');
    UPDATE Job SET Status='Closed' WHERE Job_ID = v_job;
  COMMIT;
END
```

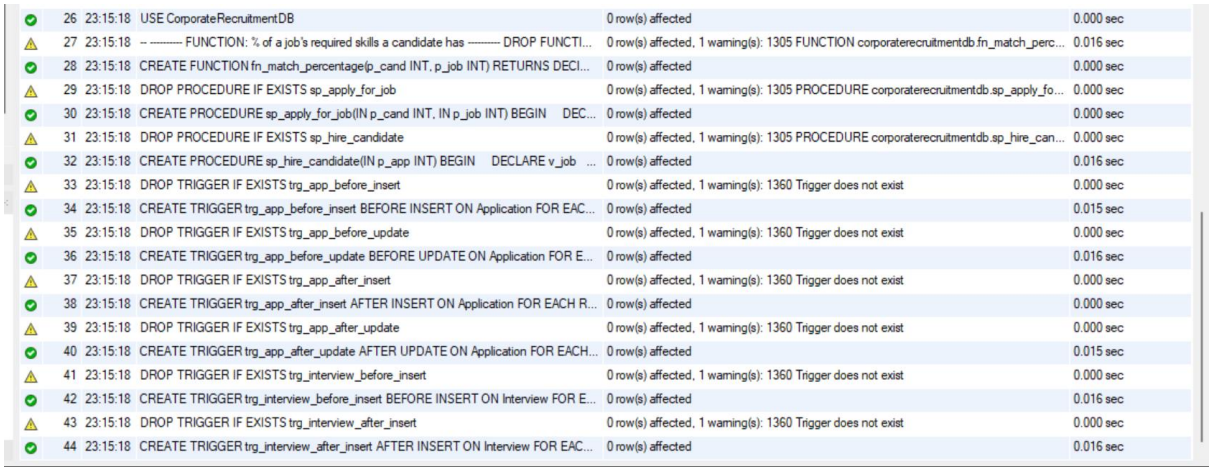


Figure 9: Script 04 output: function, procedures and six triggers created (yellow warnings are DROP … IF EXISTS on the first run)

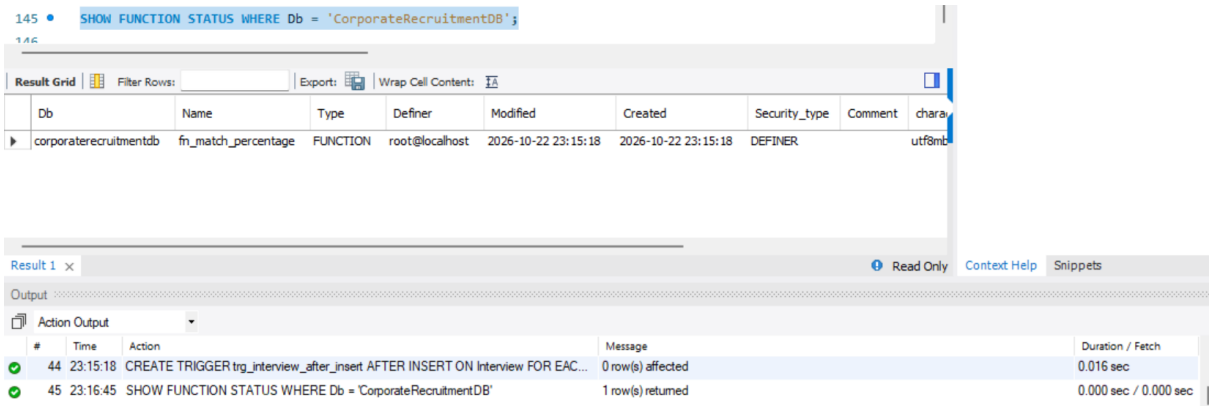


Figure 10: SHOW FUNCTION STATUS: fn_match_percentage

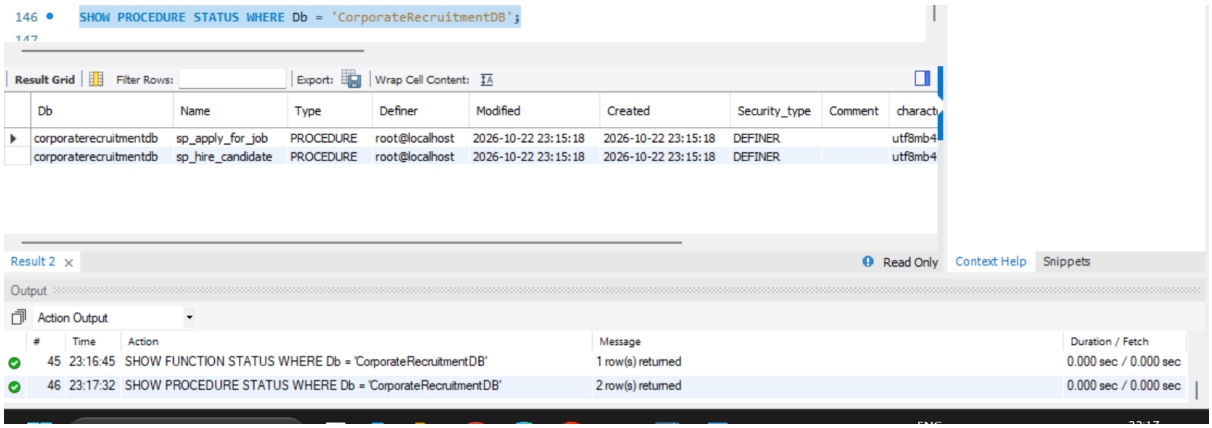


Figure 11: SHOW PROCEDURE STATUS: sp_apply_for_job and sp_hire_candidate

Trigger	Event	Table	Statement	Timing	Created	sql_mode
trg_app_before_insert	INSERT	application	BEGIN DECLARE v_status VARCHAR(10); ...	BEFORE	2026-10-22 23:15:18.89	ONLY_FULL_GROUP_BY
trg_app_after_insert	INSERT	application	BEGIN INSERT INTO Application_Status_Log ...	AFTER	2026-10-22 23:15:18.92	ONLY_FULL_GROUP_BY
trg_app_before_update	UPDATE	application	BEGIN IF OLD.Application_Status = 'Hired' A...	BEFORE	2026-10-22 23:15:18.91	ONLY_FULL_GROUP_BY
trg_app_after_update	UPDATE	application	BEGIN IF NOT (OLD.Application_Status <=> ...	AFTER	2026-10-22 23:15:18.94	ONLY_FULL_GROUP_BY
trg_interview_before_insert	INSERT	interview	BEGIN DECLARE v_app_date DATE; SELEC...	BEFORE	2026-10-22 23:15:18.95	ONLY_FULL_GROUP_BY
trg_interview_after_insert	INSERT	interview	BEGIN UPDATE Application SET Application_S...	AFTER	2026-10-22 23:15:18.97	ONLY_FULL_GROUP_BY

Figure 12: SHOW TRIGGERS: the six triggers on Application and Interview

5.5 Seed data: INSERT and UPDATE (script 05)

Sample data was loaded with INSERT statements: 4 companies, 5 departments, 10 skills, 6 candidates, 7 resumes, 6 jobs with required skills, 18 candidate-skill rows, 8 applications and 2 interviews. UPDATE statements then shortlisted and rejected applications. Because the triggers fire during seeding, the audit log is populated automatically.

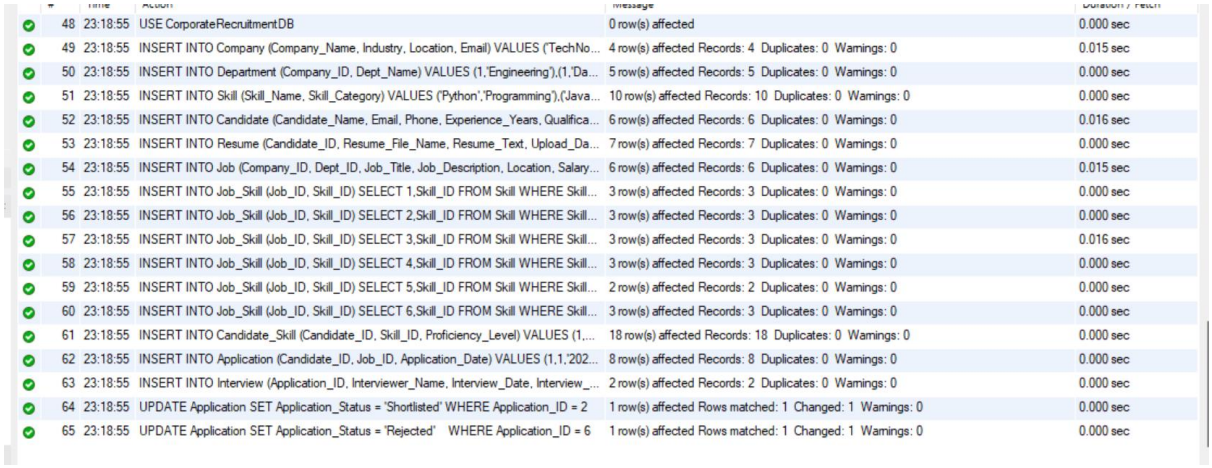


Figure 13: Seed script output: all INSERT and UPDATE statements succeeded

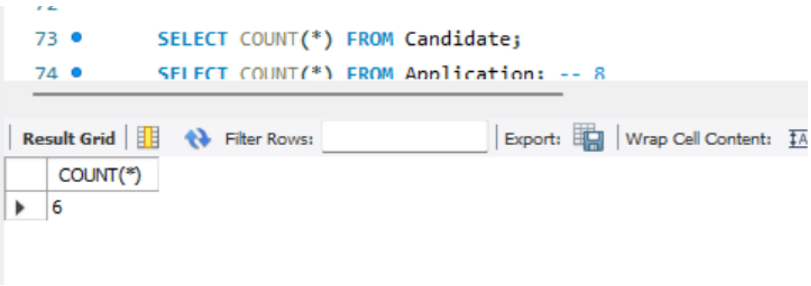


Figure 14: Row count check: 6 candidates

```
74 •  SELECT COUNT(*) FROM Application;
75 •  SELECT * FROM vw_job_summary;
```

Result Grid | Filter Rows: | Export: | Wrap

COUNT(*)
8

Figure 15: Row count check: 8 applications

```
75 •  SELECT * FROM vw_job_summary;
76 •  SELECT * FROM vw_candidate_profile;
77 •  SELECT * FROM vw_interview_schedule;
78 •  SELECT * FROM Application_Status_Log;
79
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Job_ID	Job_Title	Company_ID	Company_Name	Dept_Name	Location	Salary	Status	Total_Applications	Hired_Count
1	Backend Developer	1	TechNova Solutions	Engineering	Bengaluru	1200000.00	Open	3	0
2	Machine Learning Engineer	1	TechNova Solutions	Data Science	Bengaluru	1800000.00	Open	1	0
5	Frontend Developer	1	TechNova Solutions	Engineering	Remote	1000000.00	Open	1	0
3	Clinical Data Analyst	2	MediCare Labs	Research	Mangaluru	900000.00	Open	1	0
4	Risk Analyst	3	FinEdge Capital	Risk Analytics	Mumbai	1100000.00	Open	1	0
6	Energy Data Engineer	4	GreenGrid Energy	Operations	Pune	1300000.00	Open	1	0

Figure 16: vw_job_summary with seeded data

```
76 •  SELECT * FROM vw_candidate_profile;
77 •  SELECT * FROM vw_interview_schedule;
78 •  SELECT * FROM Application_Status_Log;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Candidate_ID	Candidate_Name	Email	Phone	Experience_Years	Qualification	Skills
1	Aarav Shetty	aarav@mail.example	9876500001	3.5	B.E. Computer Science	Java (Advanced), SQL (Advanced), Docker (
2	Diya Nair	diya@mail.example	9876500002	2.0	M.Tech AI	Python (Expert), SQL (Intermediate), Machi
3	Rohan Kamath	rohan@mail.example	9876500003	1.0	B.Sc IT	Java (Intermediate), SQL (Intermediate), Dc
4	Sneha Pai	sneha@mail.example	9876500004	5.0	MBA Finance	SQL (Advanced), Data Analysis (Advanced),
5	Karthik Rao	karthik@mail.example	9876500005	4.0	B.E. Information Science	React (Advanced), Node.js (Advanced)
6	Meera Bhat	meera@mail.example	9876500006	6.5	M.Sc Statistics	Python (Advanced), SQL (Advanced), Docke

vw_candidate_profile 4 × | Read Only | Context

Output

Action Output

#	Time	Action	Message
65	23:18:55	UPDATE Application SET Application_Status = 'Rejected' WHERE Application_ID = 6	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
66	23:20:11	SELECT COUNT(*) FROM Candidate LIMIT 0, 1000	1 row(s) returned

Figure 17: vw_candidate_profile: skills concatenated per candidate

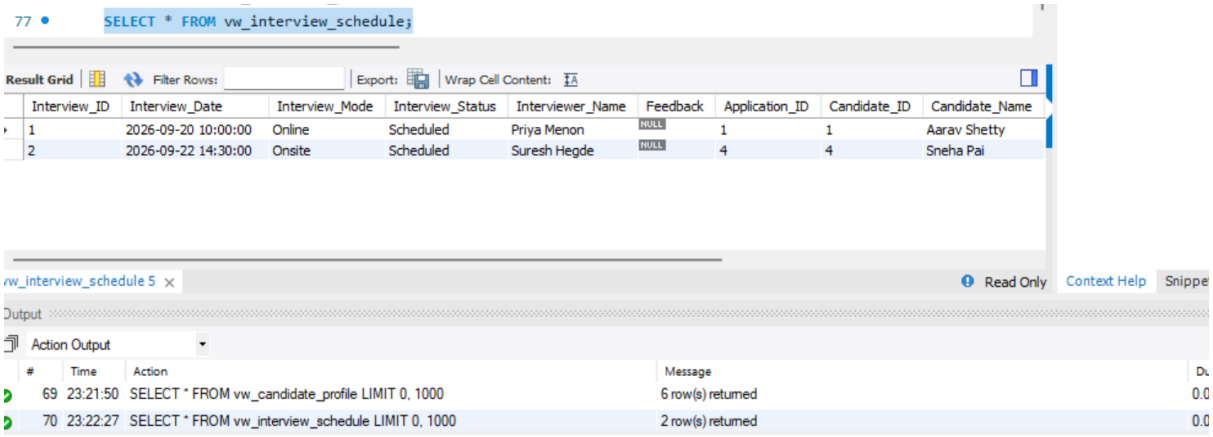


Figure 18: vw_interview_schedule

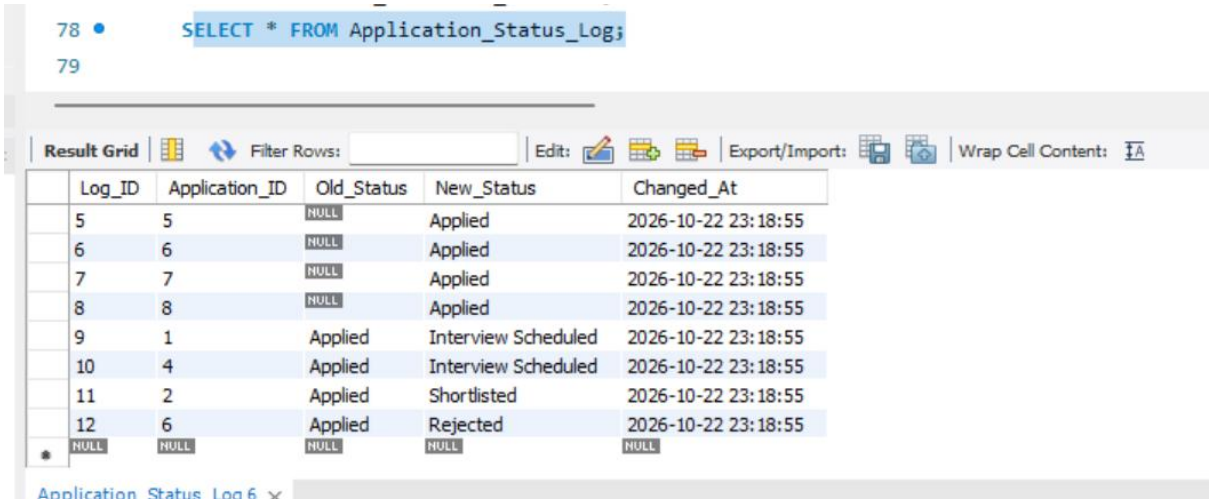


Figure 19: Application_Status_Log filled by triggers during seeding

5.6 Vector database implementation (Qdrant)

Aspect	Design
What is embedded	Resume.Resume_Text (one vector per resume) and Job_Title + Job_Description (one vector per job)
Embedding model	all-MiniLM-L6-v2 (sentence-transformers), 384 dimensions, executed locally with @xenova/transformers (ONNX)
Pre-processing	Text truncated to 2000 characters; mean pooling; vectors L2-normalised
Storage	Qdrant collections resumes (point id = Resume_ID, payload candidate_id, name) and jobs (point id = Job_ID, payload title, company_id)
Index and metric	HNSW index, cosine distance
Retrieval, candidate	Embed the latest resume (or a typed query), take the top 10 jobs, filter Status = Open in MySQL, add fn_match_percentage
Retrieval, recruiter	Embed a job description (or typed query), take the top 15 resumes, keep the best score per candidate, join vw_candidate_profile
Consistency	MySQL is the source of truth. New resumes and jobs are indexed on insert; npm run index:vectors rebuilds the index

```
export async function ensureCollections() {
  const { collections } = await client.getCollections();
  for (const name of ['resumes', 'jobs'])
    if (!collections.find((c) => c.name === name))
```

```
      await client.createCollection(name, { vectors: { size: 384, distance: 'Cosine' }
          });
}
export async function embed(text) {
  extractor ||= await pipeline('feature-extraction', 'Xenova/all-MiniLM-L6-v2');
  const out = await extractor(text.slice(0, 2000), { pooling: 'mean', normalize: true
      });
  return Array.from(out.data);
}
```

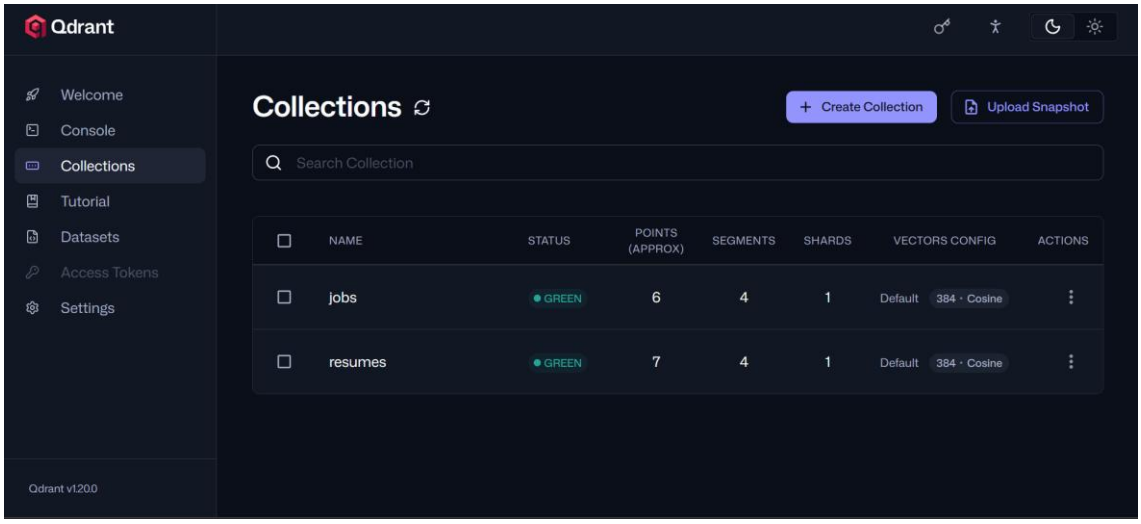


Figure 20: Qdrant dashboard: collections jobs (6 points) and resumes (7 points), 384-dimension cosine vectors

5.7 Advanced database features demonstrated

Concept	Where
Transactions and locking	sp_hire_candidate (FOR UPDATE, rollback handler); sp_apply_for_job; job posting and candidate registration use transactions in the API
Views	vw_job_summary, vw_candidate_profile, vw_interview_schedule
Stored function and procedures	fn_match_percentage, sp_apply_for_job, sp_hire_candidate
Triggers	Six triggers for integrity, audit logging and status automation
Indexing	Composite and single-column indexes on Job, Application, Candidate, Resume, Interview
Full-text search	FULLTEXT indexes ft_job and ft_resume queried with MATCH … AGAINST
Vector similarity search (vector database)	Qdrant cosine search over resume and job embeddings

6. Important Queries

The following queries (script 06) were executed on the seeded database. They cover joins, grouping, aggregation, subqueries, set-based division, full-text search, views, stored routines and DML inside a transaction.

Q1: Four-table JOIN

Lists every application with candidate, job and company.

```
SELECT a.Application_ID, ca.Candidate_Name, j.Job_Title, c.Company_Name,
    a.Application_Status
FROM Application a
JOIN Candidate ca ON ca.Candidate_ID = a.Candidate_ID
JOIN Job j        ON j.Job_ID        = a.Job_ID
JOIN Company c    ON c.Company_ID    = j.Company_ID
ORDER BY a.Application_Date DESC;
```

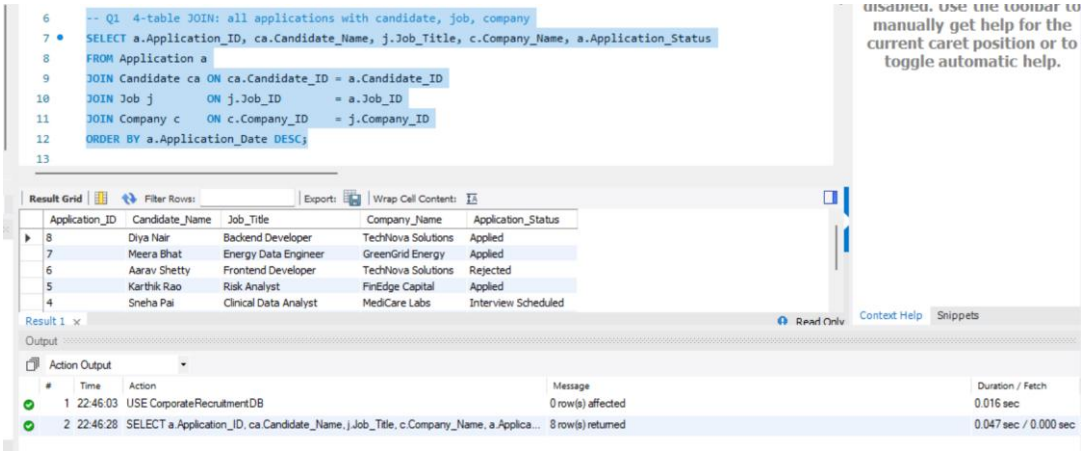


Figure 21: Q1 result

Q2: GROUP BY with HAVING

Jobs that received two or more applications.

```
SELECT j.Job_Title, COUNT(*) AS applicants
FROM Application a JOIN Job j ON j.Job_ID = a.Job_ID
GROUP BY j.Job_ID, j.Job_Title
HAVING COUNT(*) >= 2
ORDER BY applicants DESC;
```

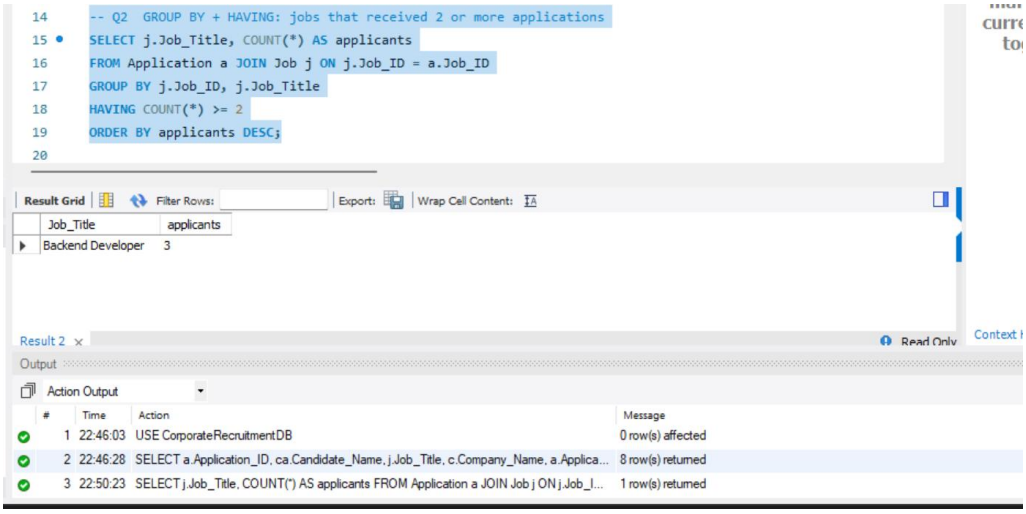


Figure 22: Q2 result

Q3: Aggregate functions

Number of openings, average and maximum salary per industry.

```
SELECT c.Industry, COUNT(*) AS openings, ROUND(AVG(j.Salary),0) AS avg_salary,
    MAX(j.Salary) AS max_salary
FROM Job j JOIN Company c ON c.Company_ID = j.Company_ID
GROUP BY c.Industry ORDER BY avg_salary DESC;
```

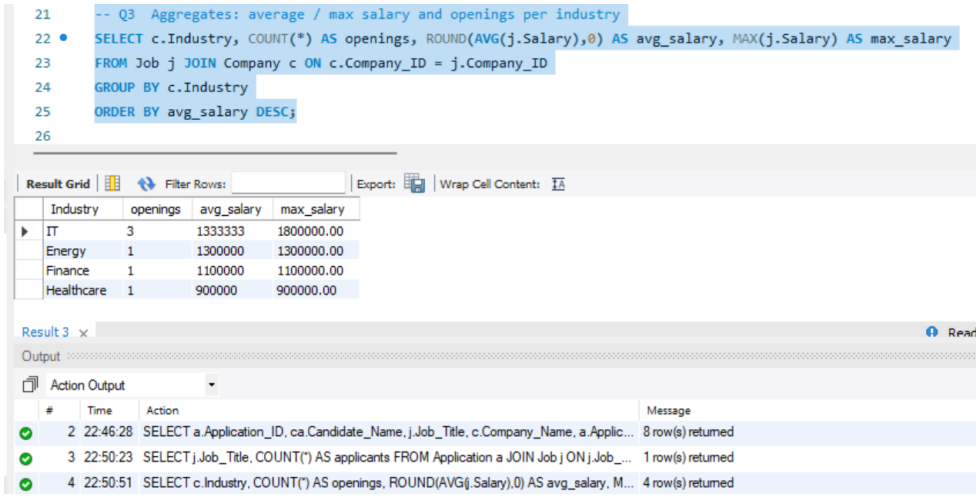


Figure 23: Q3 result

Q4: Scalar subquery

Candidates whose experience is above the overall average.

```
SELECT Candidate_Name, Experience_Years FROM Candidate
WHERE Experience_Years > (SELECT AVG(Experience_Years) FROM Candidate)
ORDER BY Experience_Years DESC;
```

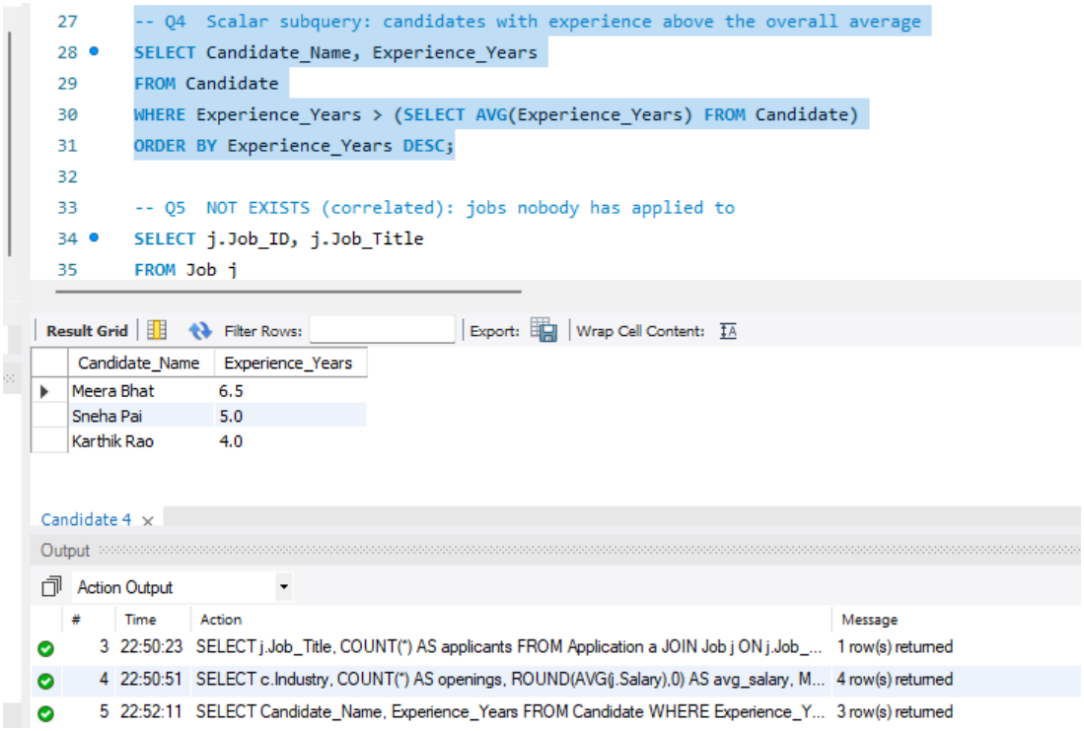


Figure 24: Q4 result

Q5: Correlated subquery with NOT EXISTS

Jobs that nobody has applied to.

```
SELECT j.Job_ID, j.Job_Title FROM Job j
WHERE NOT EXISTS (SELECT 1 FROM Application a WHERE a.Job_ID = j.Job_ID);
```

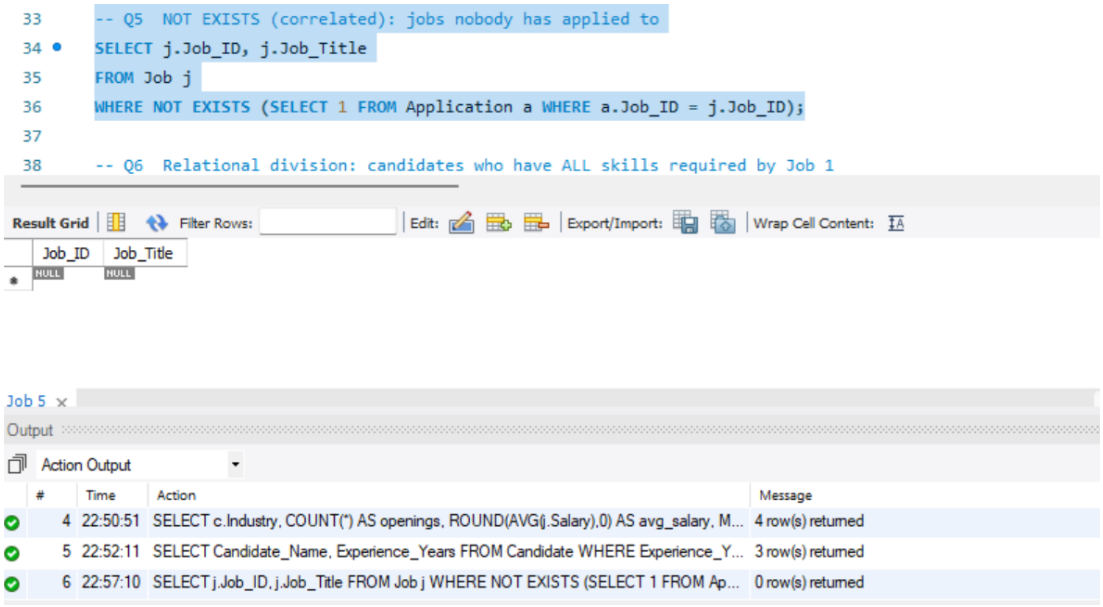


Figure 25: Q5 result

Q6: Relational division

Candidates who possess all skills required by Job 1.

```
SELECT ca.Candidate_Name
FROM Candidate ca
JOIN Candidate_Skill cs ON cs.Candidate_ID = ca.Candidate_ID
JOIN Job_Skill js       ON js.Skill_ID = cs.Skill_ID AND js.Job_ID = 1
GROUP BY ca.Candidate_ID, ca.Candidate_Name
```

```
HAVING COUNT(*) = (SELECT COUNT(*) FROM Job_Skill WHERE Job_ID = 1);
```

```
38    -- Q6  Relational division: candidates who have ALL skills required by Job 1
39    SELECT ca.Candidate_Name
40    FROM Candidate ca
41    JOIN Candidate_Skill cs ON cs.Candidate_ID = ca.Candidate_ID
42    JOIN Job_Skill js       ON js.Skill_ID = cs.Skill_ID AND js.Job_ID = 1
43    GROUP BY ca.Candidate_ID, ca.Candidate_Name
44    HAVING COUNT(*) = (SELECT COUNT(*) FROM Job_Skill WHERE Job_ID = 1);
45
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Candidate_Name
Aarav Shetty
Rohan Kamath

Result 6

Output

Action Output

#	Time	Action	Message
5	22:52:11	SELECT Candidate_Name, Experience_Years FROM Candidate WHERE Experience_Y...	3 row(s) returned
6	22:57:10	SELECT j.Job_ID, j.Job_Title FROM Job j WHERE NOT EXISTS (SELECT 1 FROM Ap...	0 row(s) returned
7	23:06:04	SELECT ca.Candidate_Name FROM Candidate ca JOIN Candidate_Skill cs ON cs.Can...	2 row(s) returned

Figure 26: Q6 result

Q7: Stored function in a query

Ranks candidates for Job 2 by skill-match percentage.

```
SELECT ca.Candidate_Name, fn_match_percentage(ca.Candidate_ID, 2) AS match_pct
FROM Candidate ca
ORDER BY match_pct DESC, ca.Experience_Years DESC LIMIT 5;
```

```
46    -- Q7  Best candidates for a job using the stored FUNCTION
47    SELECT ca.Candidate_Name, fn_match_percentage(ca.Candidate_ID, 2) AS match_pct
48    FROM Candidate ca
49    ORDER BY match_pct DESC, ca.Experience_Years DESC
50    LIMIT 5;
51
52    -- Q8  FULL-TEXT search on jobs and resumes
53    SELECT Job_ID, Job_Title, MATCH(Job_Title, Job_Description) AGAINST ('python embeddings') AS score
54    FROM Job
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

Candidate_Name	match_pct
Diya Nair	100.00
Meera Bhat	66.67
Sneha Pai	33.33
Aarav Shetty	33.33
Rohan Kamath	33.33

Result 7

Output

Action Output

#	Time	Action	Message
6	22:57:10	SELECT j.Job_ID, j.Job_Title FROM Job j WHERE NOT EXISTS (SELECT 1 FROM Ap...	0 row(s) returned
7	23:06:04	SELECT ca.Candidate_Name FROM Candidate ca JOIN Candidate_Skill cs ON cs.Can...	2 row(s) returned
8	23:09:26	SELECT ca.Candidate_Name, fn_match_percentage(ca.Candidate_ID, 2) AS match_pc...	5 row(s) returned

Figure 27: Q7 result

Q8: Full-text search

Natural-language search on jobs and boolean-mode search on resumes.

```
SELECT Job_ID, Job_Title, MATCH(Job_Title, Job_Description) AGAINST ('python
    embeddings') AS score
FROM Job
WHERE MATCH(Job_Title, Job_Description) AGAINST ('python embeddings' IN NATURAL
```

```
    LANGUAGE MODE)
ORDER BY score DESC;

SELECT Resume_ID, Candidate_ID FROM Resume
WHERE MATCH(Resume_Text) AGAINST ('+Docker +SQL' IN BOOLEAN MODE);
```

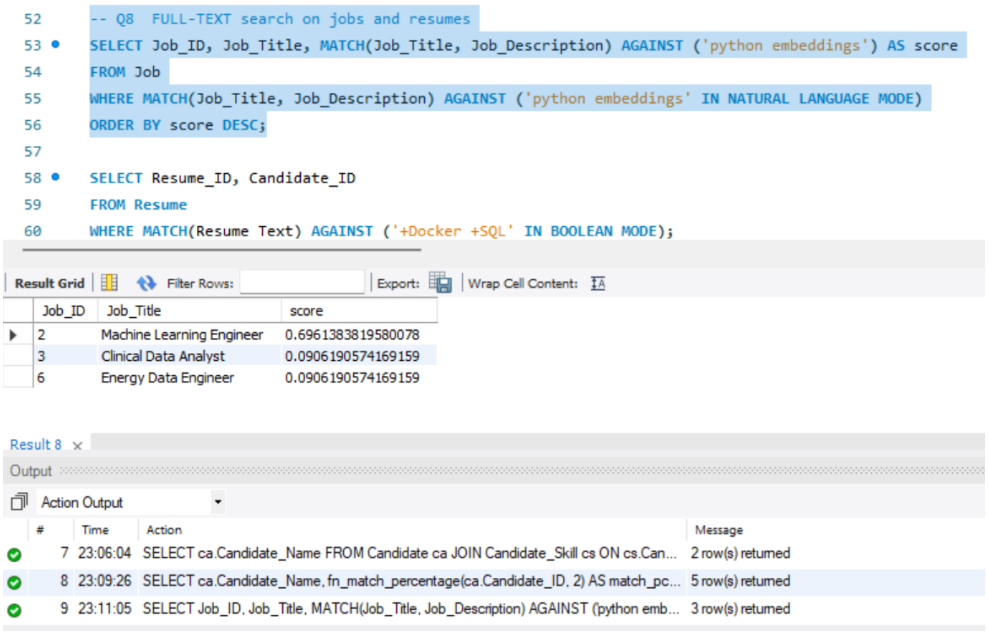


Figure 28: Q8 result: job search

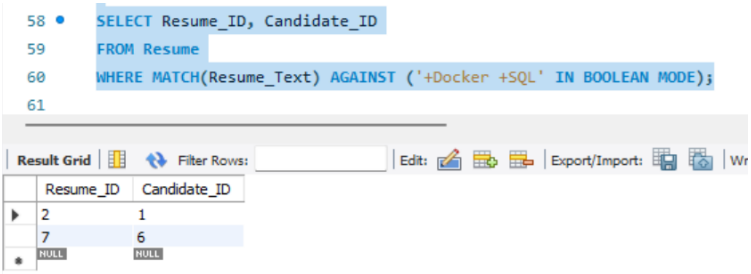


Figure 29: Q8 result: resume search

Q9: Skill demand

Most frequently required skills across all jobs.

```
SELECT s.Skill_Name, COUNT(*) AS jobs_requiring
FROM Job_Skill js JOIN Skill s ON s.Skill_ID = js.Skill_ID
GROUP BY s.Skill_ID, s.Skill_Name ORDER BY jobs_requiring DESC, s.Skill_Name;
```

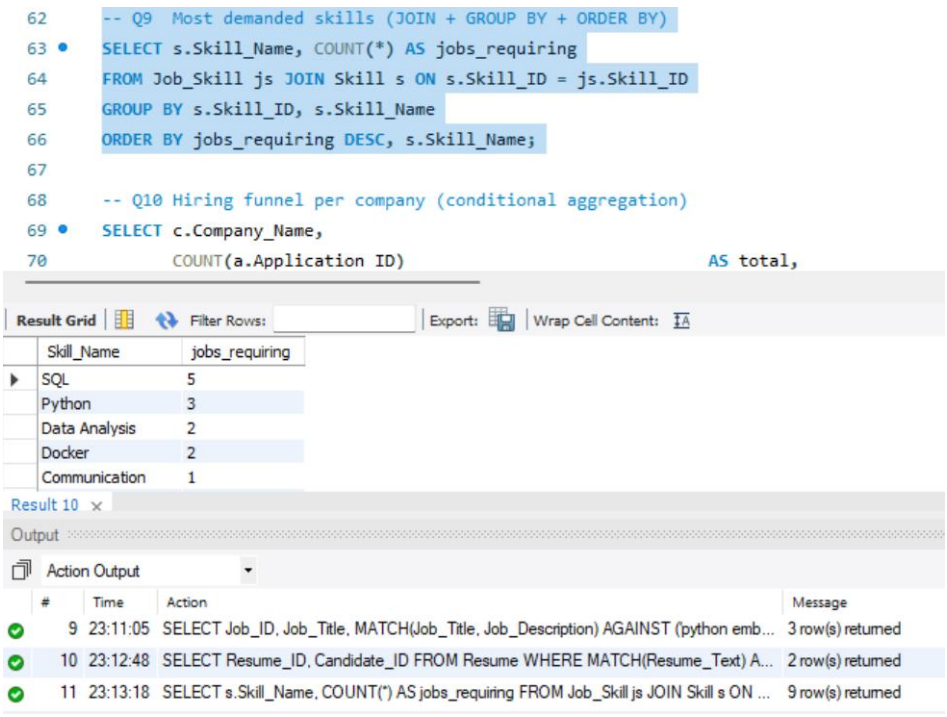


Figure 30: Q9 result

Q10: Hiring funnel with conditional aggregation

Applications, interviews, hires and hire rate per company.

```
SELECT c.Company_Name, COUNT(a.Application_ID) AS total,
       SUM(a.Application_Status = 'Interview Scheduled') AS interviewing,
       SUM(a.Application_Status = 'Hired') AS hired,
       ROUND(100*SUM(a.Application_Status='Hired')/NULLIF(COUNT(a.Application_ID),0),1) AS
hire_rate_pct
FROM Company c
LEFT JOIN Job j          ON j.Company_ID = c.Company_ID
LEFT JOIN Application a ON a.Job_ID     = j.Job_ID
GROUP BY c.Company_ID, c.Company_Name;
```

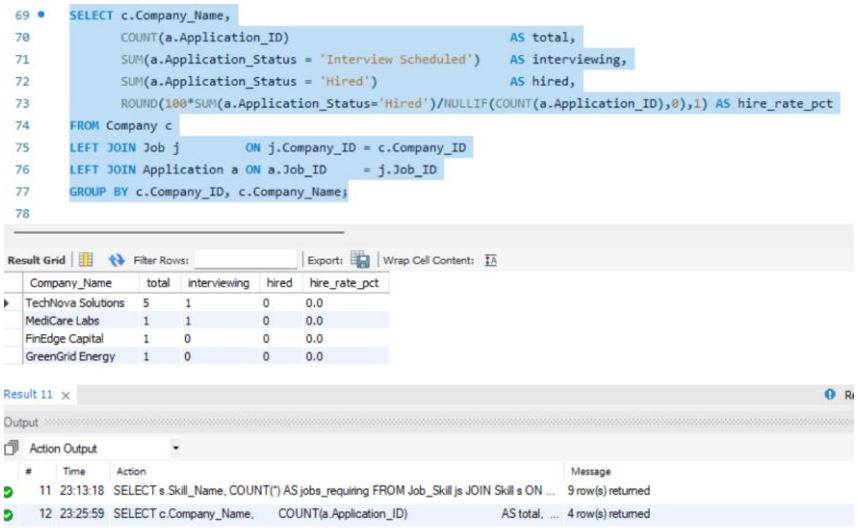


Figure 31: Q10 result

Q11: LEFT JOIN anti-join

Candidates who have never applied.

```
SELECT ca.Candidate_Name
```

```
FROM Candidate ca LEFT JOIN Application a ON a.Candidate_ID = ca.Candidate_ID
WHERE a.Application_ID IS NULL;
```

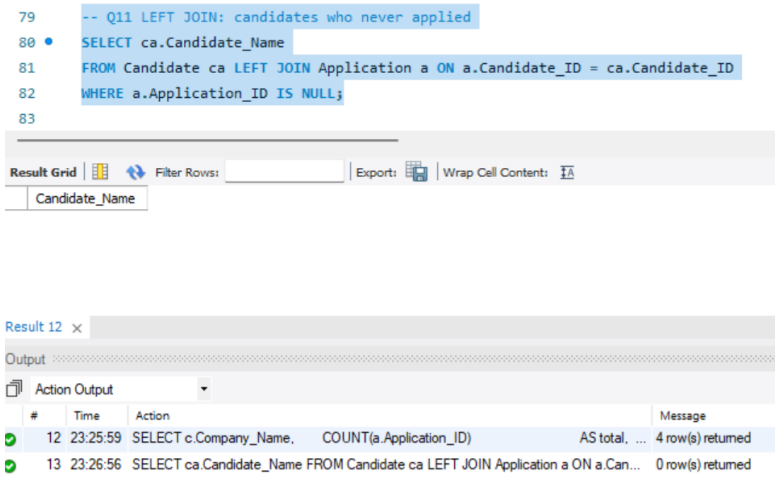


Figure 32: Q11 result

Q12: Querying the views

Views used like ordinary tables.

```
SELECT * FROM vw_job_summary ORDER BY Total_Applications DESC;
SELECT * FROM vw_candidate_profile WHERE Skills LIKE '%SQL%';
SELECT * FROM vw_interview_schedule;
```

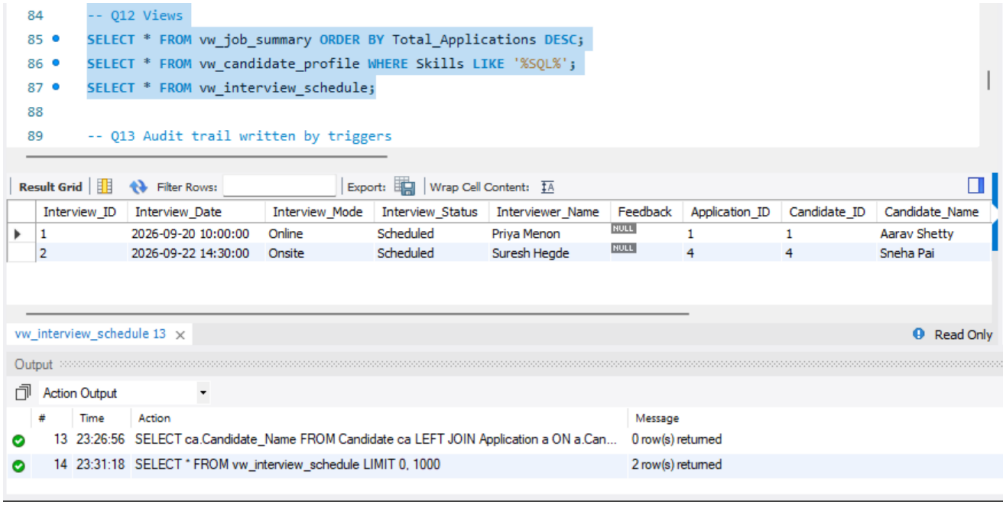


Figure 33: Q12 result

Q13: Audit trail written by triggers

Every status change is recorded without any application code.

```
SELECT * FROM Application_Status_Log ORDER BY Log_ID DESC LIMIT 10;
```

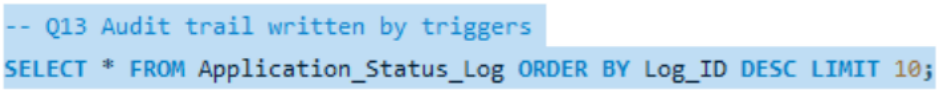


Figure 34: Q13 statement

Log_ID	Application_ID	Old_Status	New_Status	Changed_At
12	6	Applied	Rejected	2026-10-22 23:18:55
11	2	Applied	Shortlisted	2026-10-22 23:18:55
10	4	Applied	Interview Scheduled	2026-10-22 23:18:55
9	1	Applied	Interview Scheduled	2026-10-22 23:18:55
8	8	NULL	Applied	2026-10-22 23:18:55
7	7	NULL	Applied	2026-10-22 23:18:55
6	6	NULL	Applied	2026-10-22 23:18:55
5	5	NULL	Applied	2026-10-22 23:18:55
4	4	NULL	Applied	2026-10-22 23:18:55
3	3	NULL	Applied	2026-10-22 23:18:55
NULL	NULL	NULL	NULL	NULL

Figure 35: Q13 result

Q14: Stored procedures: apply and hire

A new application is created, then Candidate 1 is hired. The procedure rejects the other applicants and closes the job atomically.

```
CALL sp_apply_for_job(4, 1);
CALL sp_hire_candidate(1);
SELECT Application_ID, Application_Status FROM Application WHERE Job_ID = 1;
SELECT Job_ID, Status FROM Job WHERE Job_ID = 1;
```

```
92    -- Q14 Procedures: apply, then hire (atomic)
93 •  CALL sp_apply_for_job(4, 1);       -- Sneha applies to Backend Developer
94 •  CALL sp_hire_candidate(1);          -- Hire Aarav; others rejected; job closed
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

Result	Job_ID
Hired	1

esult 15 ×

utput

Action Output

#	Time	Action	Message
32	23:58:18	INSERT INTO Interview (Application_ID, Interviewer_Name, Interview_Date, Interview_...	1 row(s) affected
33	23:58:44	SELECT Application_Status FROM Application WHERE Application_ID = 2 LIMIT 0, 1000	1 row(s) returned
34	23:59:27	DELETE FROM Candidate WHERE Candidate_ID = @cid	0 row(s) affected
35	00:00:04	SELECT (SELECT COUNT(*) FROM Resume WHERE Candidate_ID=@cid) AS resume...	1 row(s) returned
36	00:10:20	CALL sp_hire_candidate(1)	1 row(s) returned

Figure 36: Q14: statuses of all applications to Job 1 after hiring

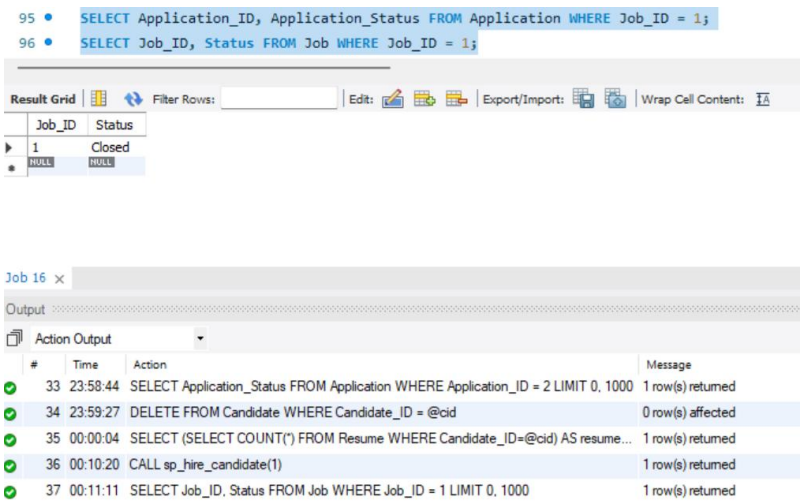


Figure 37: Q14: Job 1 is now Closed

Q15: INSERT, UPDATE and DELETE inside a transaction

Demonstrates DML and rollback; the seed data is left untouched.

```
START TRANSACTION;
  INSERT INTO Skill (Skill_Name, Skill_Category) VALUES ('Kubernetes','DevOps');
  UPDATE Candidate SET Experience_Years = Experience_Years + 1 WHERE Candidate_ID = 3;
  DELETE FROM Skill WHERE Skill_Name = 'Kubernetes';
ROLLBACK;
```

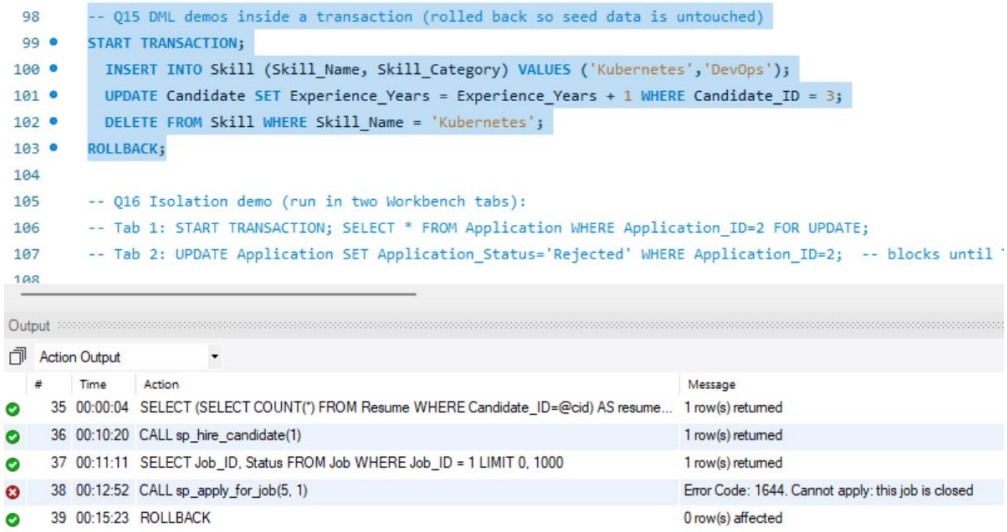


Figure 38: Q15 output: all statements ran, then ROLLBACK

7. System Architecture

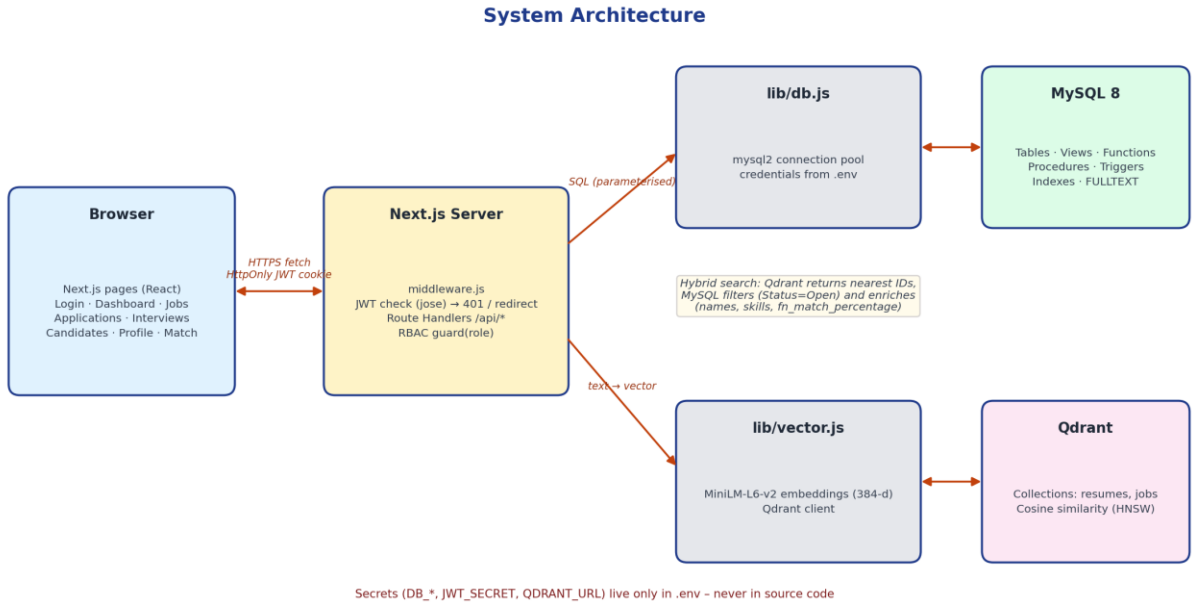


Figure 39: System architecture: browser, Next.js server, MySQL and Qdrant

7.1 Components

Component	Responsibility
Browser (React pages)	Login, dashboard, jobs, applications, interviews, candidates, profile, semantic match
middleware.js	Verifies the JWT cookie on every page and API request; returns 401 or redirects to /login
Route Handlers (/api/*)	Business logic; each handler calls guard(role) for role-based access and uses parameterised SQL only
lib/db.js	mysql2 connection pool configured from environment variables
lib/vector.js	Local embedding model and Qdrant client
MySQL 8	Tables, constraints, views, function, procedures, triggers, indexes
Qdrant	Vector index of resumes and jobs

7.2 API endpoints

Method	Endpoint	Roles	Purpose
POST	`/api/auth/login, /logout`	All	Authenticate (bcrypt compare) and set an HttpOnly JWT cookie
POST	`/api/auth/register`	Public	Create Candidate and Users rows in one transaction
GET / POST	`/api/jobs`	All / Recruiter	Full-text, location, type and salary search with skill-match; post a job with skills
PATCH / DELETE	`/api/jobs/[id]`	Recruiter, Admin	Close or reopen a job; delete (Admin only)

Method	Endpoint	Roles	Purpose
GET / POST / PATCH	`/api/applications`	Role-scoped	List; apply (calls sp_apply_for_job); change status
POST	`/api/applications/hire`	Recruiter, Admin	Calls sp_hire_candidate (transaction)
GET / POST / PATCH	`/api/interviews`	Role-scoped	View vw_interview_schedule; schedule; complete or cancel
GET	`/api/candidates`	Recruiter, Admin	Filter vw_candidate_profile by skill and experience
GET / POST	`/api/profile`	Candidate	Profile, skills, resume upload (also indexed in Qdrant)
GET	`/api/search/semantic`	Role-aware	Vector search in Qdrant enriched from MySQL
GET	`/api/dashboard`	Recruiter, Admin	Aggregate queries and views

7.3 Authentication, authorisation and configuration

- **Authentication:** passwords are stored as bcrypt hashes (cost 10). A successful login issues a signed JWT (8 hours) in an HttpOnly, SameSite cookie, so scripts in the browser cannot read it.
- **Authorisation:** three roles. Recruiters only see and change data of their own company; candidates only see their own applications and interviews.
- **SQL injection:** every query uses placeholders; no user input is concatenated into SQL.
- **Configuration:** DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET and QDRANT_URL are read from the .env file, which is excluded from Git. Nothing sensitive is hard-coded.

Capability	Admin	Recruiter	Candidate
View all companies' data	Yes	Own company	Own data
Post, close, reopen jobs	Close / reopen	Own company	No
Delete a job	Yes	No	No
Apply to a job	No	No	Yes
Change status, schedule interviews, hire	Yes	Own company	No
Dashboard and candidate list	Yes	Yes	No
Edit profile, skills, resumes	No	No	Yes
Recommended jobs (semantic)	—	—	Yes
Matching candidates (semantic)	Yes	Yes	No

8. Application Screenshots

8.1 Setup and login

Demo accounts are created by a script that stores only bcrypt hashes in the Users table.

```
PS C:\Sumedh\DBMS LAB\recruitment-app\recruitment-app> npm run seed:use

> recruitment-app@1.0.0 seed:users
> node --env-file=.env scripts/seed-users.mjs

recruiter -> recruiter_technova / Recruit@123
recruiter -> recruiter_medicare / Recruit@123
recruiter -> recruiter_finedge / Recruit@123
recruiter -> recruiter_greengrid / Recruit@123
candidate -> aarav / Cand@123
candidate -> diya / Cand@123
candidate -> rohan / Cand@123
candidate -> sneha / Cand@123
candidate -> karthik / Cand@123
candidate -> meera / Cand@123
admin -> admin / Admin@123
PS C:\Sumedh\DBMS LAB\recruitment-app\recruitment-app>
```

Figure 40: npm run seed:users: recruiter, candidate and admin accounts created

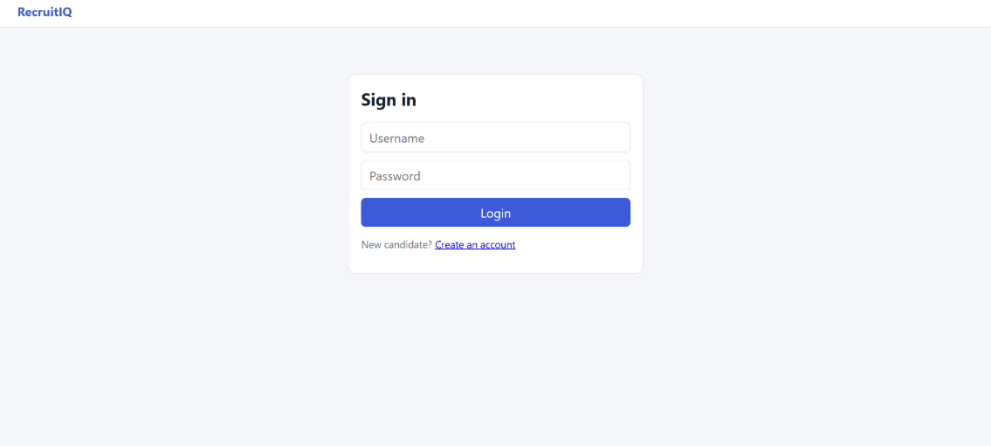


Figure 41: Login page

8.2 Recruiter flow

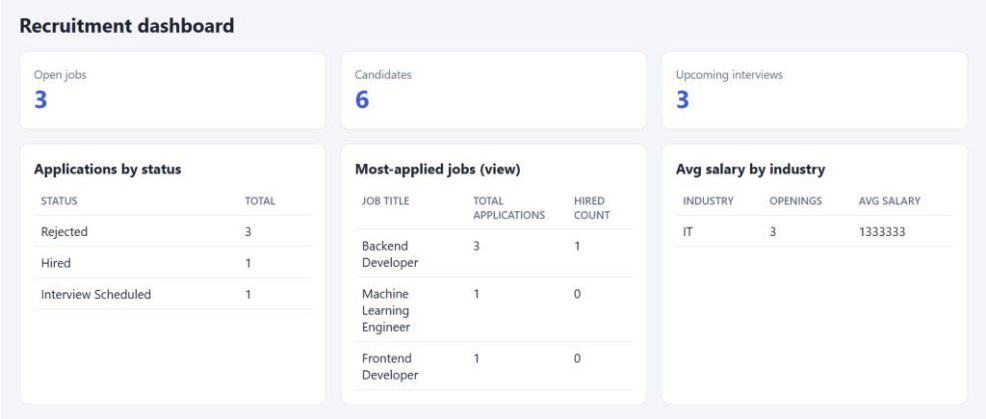
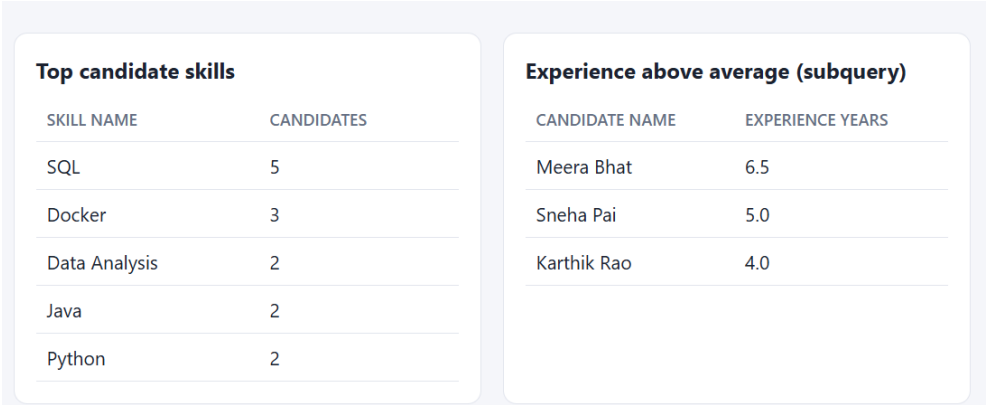


Figure 42: Recruiter dashboard: totals and applications by status



Top candidate skills

SKILL NAME	CANDIDATES
SQL	5
Docker	3
Data Analysis	2
Java	2
Python	2

Experience above average (subquery)

CANDIDATE NAME	EXPERIENCE YEARS
Meera Bhat	6.5
Sneha Pai	5.0
Karthik Rao	4.0

Figure 43: Recruiter dashboard: most-applied jobs (from vw_job_summary), average salary by industry, top skills

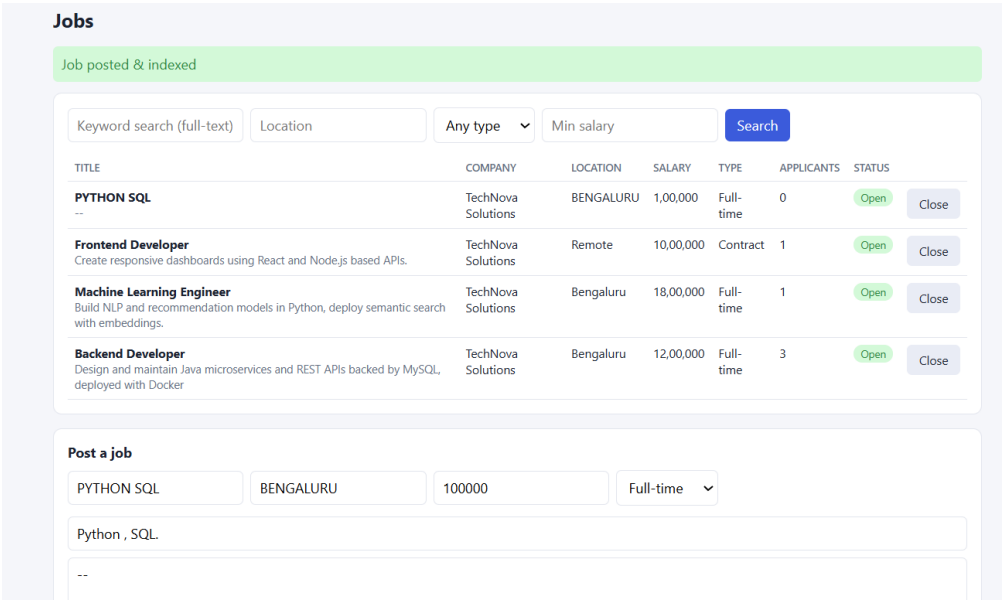


Figure 44: Posting a new job with required skills (job and Job_Skill rows inserted in one transaction, then indexed in Qdrant)

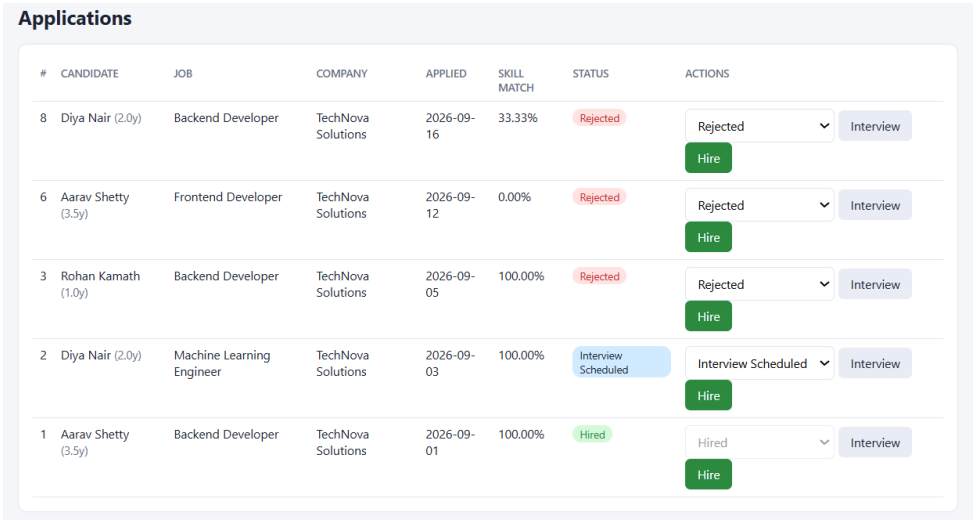


Figure 45: Applications screen with status, skill-match percentage and actions

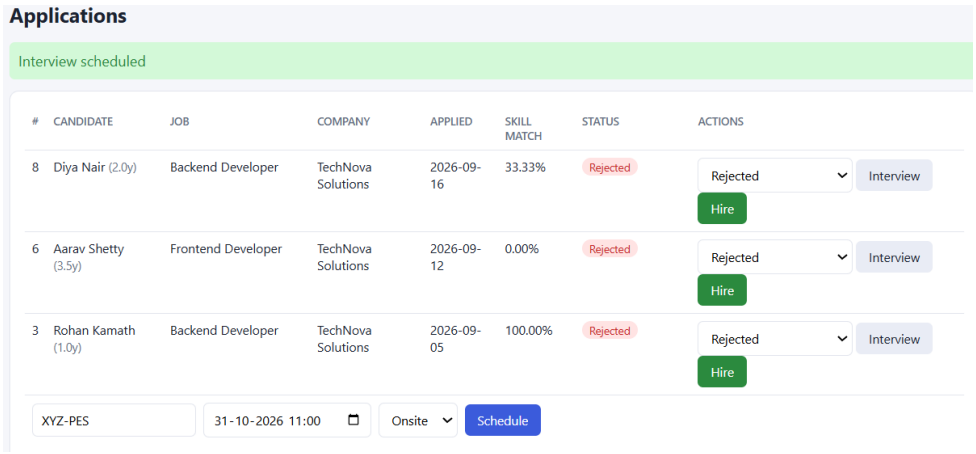


Figure 46: Scheduling an interview for an application

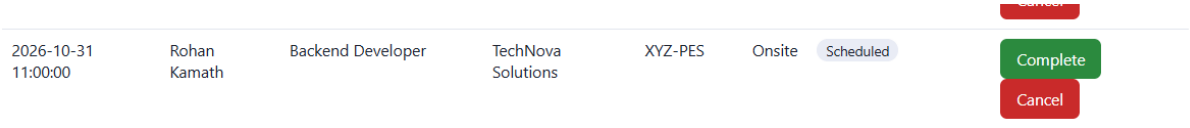


Figure 47: Interview schedule page

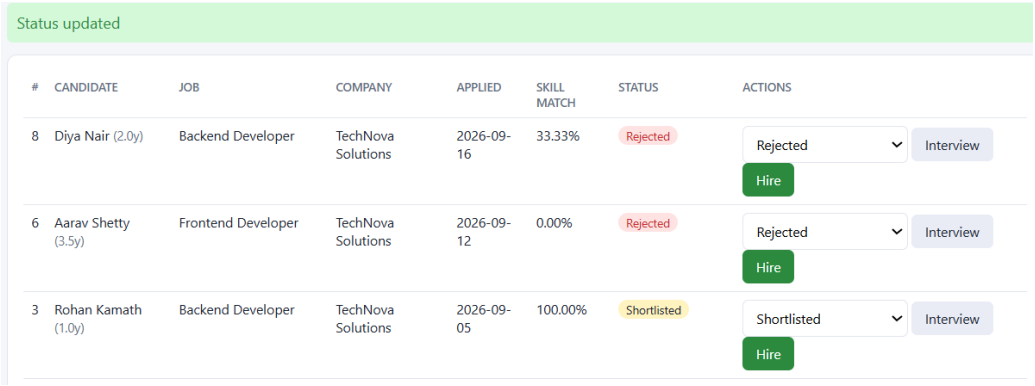


Figure 48: Status changed to Shortlisted; the trigger records the change in the audit log

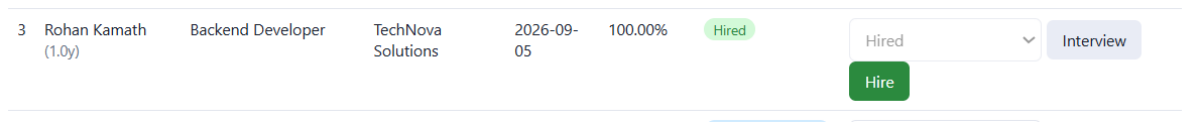


Figure 49: Candidate hired through sp_hire_candidate

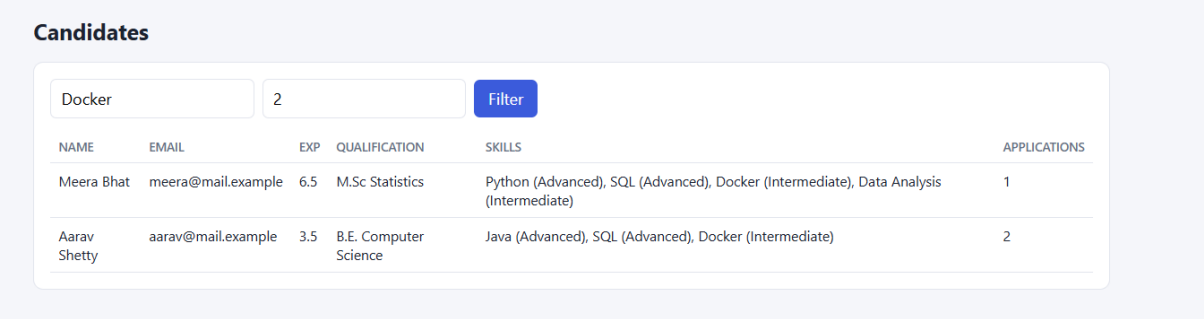


Figure 50: Filtering candidates by skill and experience

8.3 Candidate flow

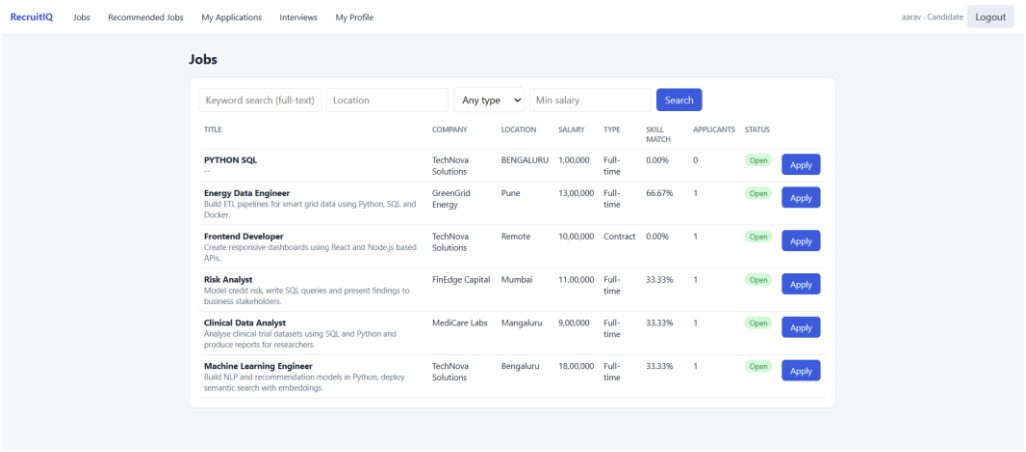


Figure 51: Jobs a candidate can apply to, with the skill-match percentage from fn_match_percentage

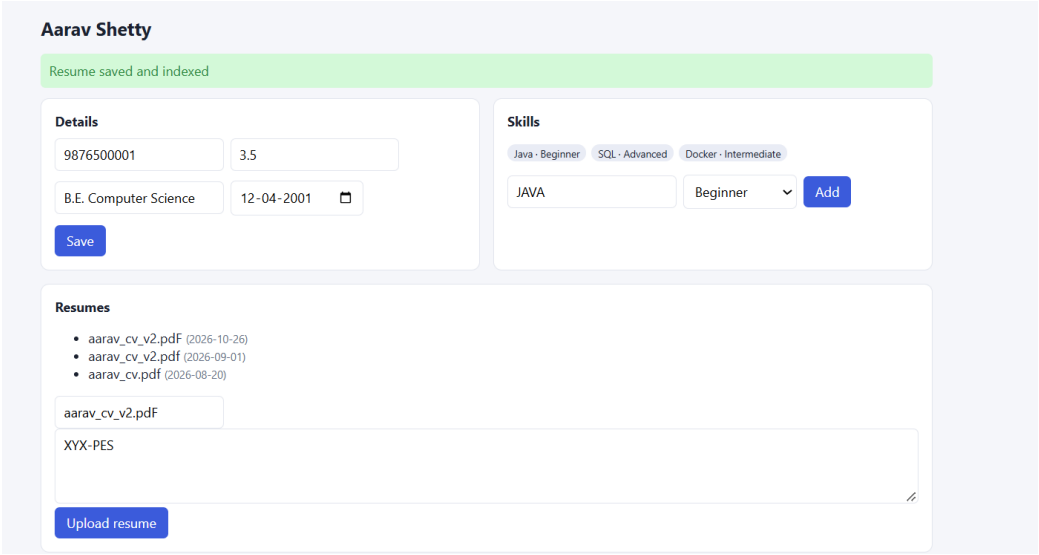


Figure 52: Candidate profile: details, skills and resume upload

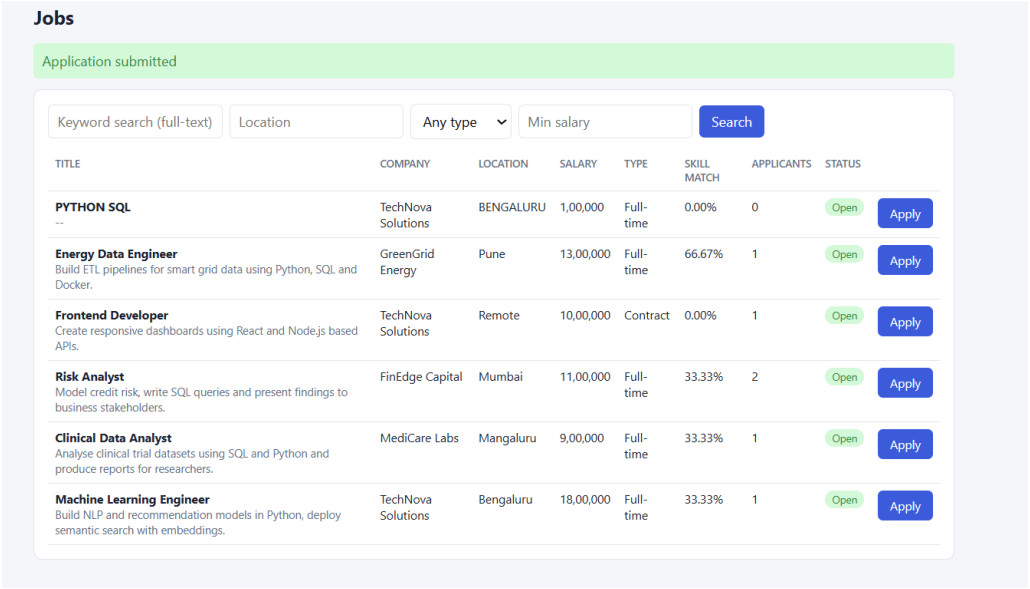


Figure 53: Applying to a job: application submitted

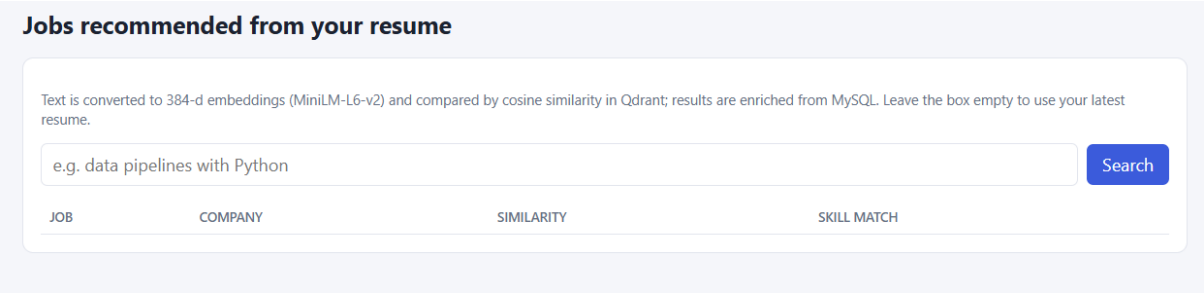


Figure 54: Recommended Jobs page (semantic search over the candidate's resume)

Add here: screenshot of the semantic search results (Recruiter: Semantic Match with a job selected; Candidate: Recommended Jobs after pressing Search).

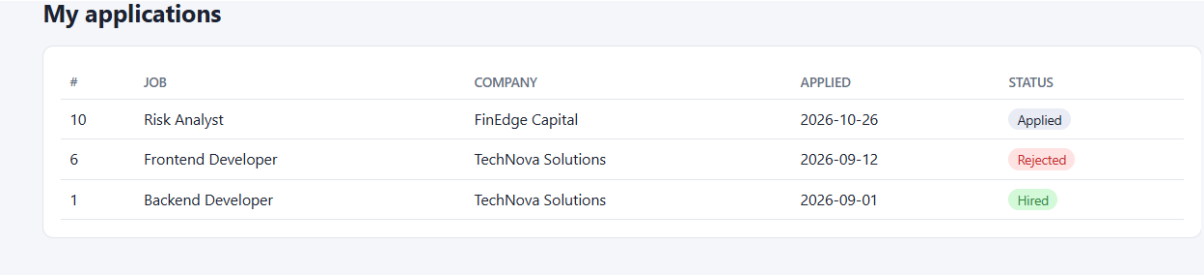


Figure 55: My applications

8.4 Security checks

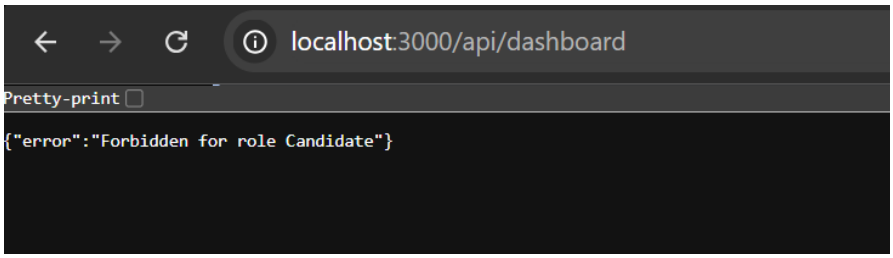


Figure 56: A candidate calling a recruiter-only API receives 403 Forbidden

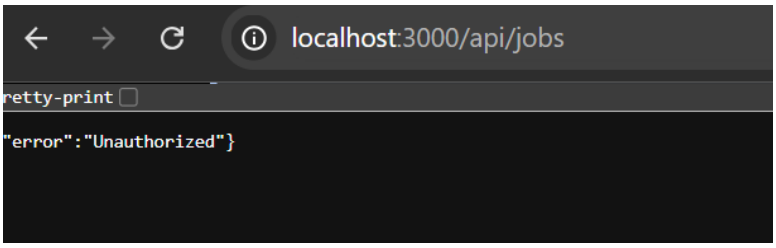


Figure 57: A logged-out request to a protected API receives 401 Unauthorized

9. Testing Results

Database-level tests were executed from MySQL Workbench (script 07). Negative tests are expected to fail with the stated error; positive tests must succeed. Application-level tests were executed in the browser.

9.1 Summary of test cases

ID	Test	Expected	Actual	Result
T1	Candidate inserted without a name	Rejected (NOT NULL)	Error 1364: Candidate_Name has no default value	**Pass**
T2	Duplicate candidate email	Rejected (UNIQUE)	Error 1062: duplicate entry for key candidate.Email	**Pass**
T3	Negative experience	Rejected (CHECK)	Error 3819: CHK_Experience violated	**Pass**
T4	Phone shorter than 10 characters	Rejected (CHECK)	Error 3819: CHK_Phone violated	**Pass**
T5	Negative salary	Rejected (CHECK)	Error 3819: CHK_Salary violated	**Pass**
T6	Invalid job type	Rejected (CHECK)	Error 3819: CHK_Job_Type violated	**Pass**
T7	Job for a non-existent company	Rejected (FK)	Error 1452: foreign key constraint fails	**Pass**
T8	Same candidate applies twice to a job	Rejected (UNIQUE)	Error 1062: duplicate entry for UQ_Candidate_Job	**Pass**
T9	Invalid application status	Rejected (CHECK)	Error 3819: CHK_App_Status violated	**Pass**
T10	Apply without a resume	Rejected (trigger)	Error 1644: Upload a resume before applying	**Pass**
T11	Interview dated before the application	Rejected (trigger)	Error 1644: Interview date cannot be before application date	**Pass**
T12	Delete a company that has jobs	Rejected (FK RESTRICT)	Error 1451: cannot delete a parent row	**Pass**
T13	Hire an application that is only Applied	Rejected, nothing changed	Error 1644: Only shortlisted / interviewed applications can be hired	**Pass**
T14	Apply to a closed job	Rejected (trigger)	Error 1644: Cannot apply: this job is closed	**Pass**
P1	Schedule an interview	Application moves to Interview Scheduled	Status of Application 2 became Interview Scheduled; log row added	**Pass**
P2	Delete a candidate (ON DELETE CASCADE)	Resumes and skills removed	Counts returned 0 and 0; the DELETE affected 0 rows because @cid was not set	**Re-run**
P3	Hire through sp_hire_candidate	Hired; others Rejected; job Closed	Application 1 Hired, Applications 3 and 8 Rejected, Job 1 Closed; 3 audit rows	**Pass**
A1	Candidate calls the dashboard API	403 Forbidden	Forbidden for role Candidate	**Pass**
A2	Recruiter changes another company's application	404 Not found	Screenshot to be added	**Pending**
A3	Logged-out request to a protected API	401 Unauthorized	Unauthorized	**Pass**

ID	Test	Expected	Actual	Result
A4	Login with a wrong password	401 Invalid credentials	Screenshot to be added	**Pending**
V1	Recruiter semantic search “NLP recommendation”	Machine-learning resume ranked first	Screenshot to be added	**Pending**
V2	Candidate Recommended Jobs from latest resume	Best-matching jobs on top	Screenshot to be added	**Pending**

> Outcome: all 14 negative database tests were rejected with the intended error, and the integrity checks and triggers behave as designed. The remaining items marked Pending or Re-run need a screenshot or a repeat run before final submission.

9.2 Evidence: negative tests (T1 to T14)

Each pair shows the statement executed and the error returned by MySQL.

T1

```
-- T1  NOT NULL: candidate without name
INSERT INTO Candidate (Email) VALUES ('x@mail.example');
```

Figure 58: T1: statement



Figure 59: T1: error returned

T2

```
-- T2  UNIQUE: duplicate email
INSERT INTO Candidate (Candidate_Name, Email) VALUES ('Dup','aarav@mail.example');
```

Figure 60: T2: statement

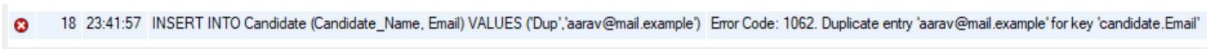


Figure 61: T2: error returned

T3

```
-- T3  CHECK: negative experience
INSERT INTO Candidate (Candidate_Name, Email, Experience_Years) VALUES ('Neg','neg@mail.example',-1);
```

Figure 62: T3: statement

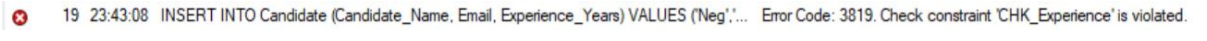


Figure 63: T3: error returned

T4

```
-- T4  CHECK: phone shorter than 10 chars
INSERT INTO Candidate (Candidate_Name, Email, Phone) VALUES ('Short','s@mail.example','12345');
```

Figure 64: T4: statement



Figure 65: T4: error returned

T5

```
-- T5  CHECK: negative salary
INSERT INTO Job (Company_ID, Job_Title, Salary) VALUES (1,'Bad Job',-100);
```

Figure 66: T5: statement

```
21  23:44:40  INSERT INTO Job (Company_ID, Job_Title, Salary) VALUES (1,'Bad Job',-100)   Error Code: 3819. Check constraint 'CHK_Salary' is violated.
```

Figure 67: T5: error returned

T6

```
-- T6  CHECK: invalid job type
INSERT INTO Job (Company_ID, Job_Title, Job_Type) VALUES (1,'Bad Type','Weekend');
```

Figure 68: T6: statement

```
22  23:46:36  INSERT INTO Job (Company_ID, Job_Title, Job_Type) VALUES (1,'Bad Type','Weeke...   Error Code: 3819. Check constraint 'CHK_Job_Type' is violated.
```

Figure 69: T6: error returned

T7

```
-- T7  FK: job for non-existent company
INSERT INTO Job (Company_ID, Job_Title) VALUES (999,'Ghost Job')
```

Figure 70: T7: statement

```
23  23:48:11  INSERT INTO Job (Company_ID, Job_Title) VALUES (999,'Ghost Job')   Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails (`corpo...   0.015 sec
```

Figure 71: T7: error returned

T8

```
-- T8  UNIQUE (composite): same candidate applies twice
INSERT INTO Application (Candidate_ID, Job_ID, Application_Date) VALUES (1,1,CURDATE());
```

Figure 72: T8: statement

```
24  23:48:55  INSERT INTO Application (Candidate_ID, Job_ID, Application_Date) VALUES (1,1,CU...   Error Code: 1062. Duplicate entry '1-1' for key 'application.UQ_Candidate_Job'   0.015 sec
```

Figure 73: T8: error returned

T9

```
-- T9  CHECK: invalid application status
UPDATE Application SET Application_Status = 'Maybe' WHERE Application_ID = 2;
```

Figure 74: T9: statement

```
25  23:50:24  UPDATE Application SET Application_Status = 'Maybe' WHERE Application_ID = 2   Error Code: 3819. Check constraint 'CHK_App_Status' is violated.
```

Figure 75: T9: error returned

T10

```
-- T10 TRIGGER: apply without a resume (add candidate first)
INSERT INTO Candidate (Candidate_Name, Email) VALUES ('No Resume','nr@mail.example');
```

Figure 76: T10: a candidate without any resume is inserted (succeeds)

```
26  23:52:03  INSERT INTO Candidate (Candidate_Name, Email) VALUES ('No Resume','nr@mail.exa...   1 row(s) affected
```

Figure 77: T10: insert output

```
INSERT INTO Application (Candidate_ID, Job_ID, Application_Date)
VALUES (LAST_INSERT_ID(), 2, CURDATE());
```

Figure 78: T10: application for that candidate

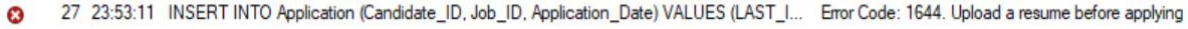


Figure 79: T10: trigger error – resume required

T11

```
-- T11 TRIGGER: interview before application date
INSERT INTO Interview (Application_ID, Interviewer_Name, Interview_Date)
VALUES (2,'Test','2020-01-01 10:00:00');
```

Figure 80: T11: statement



Figure 81: T11: error returned

T12

```
-- T12 FK RESTRICT: delete company that has jobs
DELETE FROM Company WHERE Company_ID = 1;
```

Figure 82: T12: statement



Figure 83: T12: error returned

T13

```
-- T13 PROCEDURE: hire an application that is only 'Applied'
CALL sp_hire_candidate(3);
```

Figure 84: T13: statement



Figure 85: T13: error returned

T14

```
-- T14 TRIGGER: apply to a closed job (after Q14 closed job 1)  EXPECT ERROR 1644
CALL sp_apply_for_job(5, 1);
```

Figure 86: T14: statement



Figure 87: T14: error returned

9.3 Evidence: positive tests and audit log

P1: interview scheduling advances the application

```
-- P1  Interview insert auto-advances status
INSERT INTO Interview (Application_ID, Interviewer_Name, Interview_Date, Interview_Mode)
VALUES (2,'Anita Rao','2026-10-05 11:00:00','Phone');
```

Figure 88: P1: INSERT INTO Interview

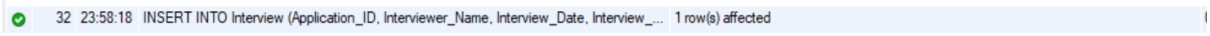


Figure 89: P1: output

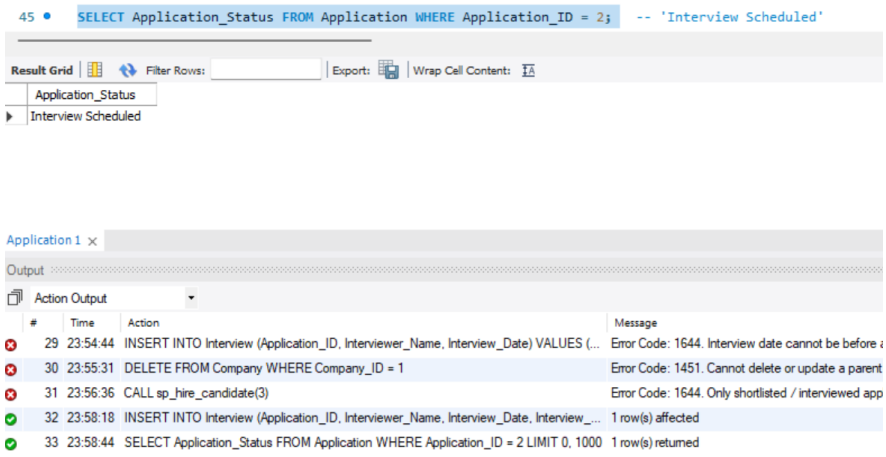


Figure 90: P1: Application 2 is now Interview Scheduled

P2: cascade delete inside a transaction

```
-- P2  ON DELETE CASCADE: deleting a candidate removes their resumes, skills and login
START TRANSACTION;
  INSERT INTO Candidate (Candidate_Name, Email) VALUES ('Temp Person','temp@mail.example');
  SET @cid = LAST_INSERT_ID();
  INSERT INTO Resume (Candidate_ID, Resume_File_Name, Upload_Date) VALUES (@cid,'t.pdf',CURDATE());
  INSERT INTO Candidate_Skill VALUES (@cid,1,'Beginner');
  DELETE FROM Candidate WHERE Candidate_ID = @cid;
```

Figure 91: P2: transaction statements



Figure 92: P2: output

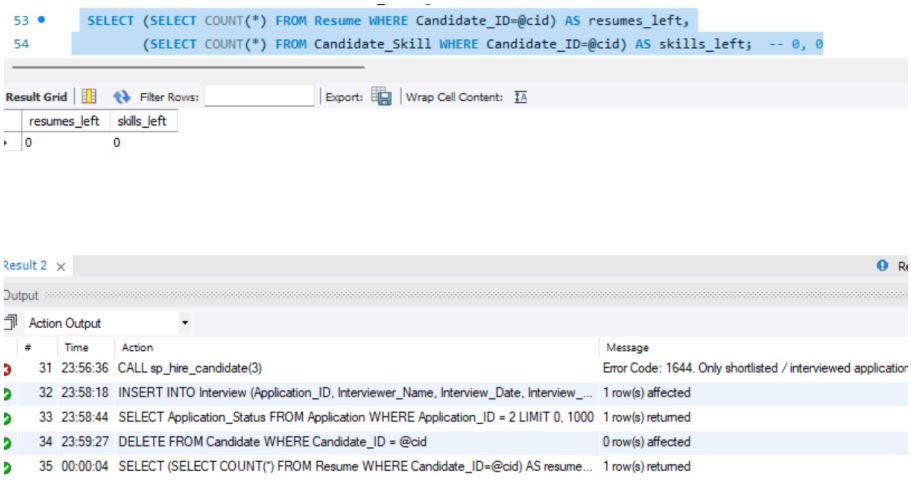


Figure 93: P2: result

> Re-run P2 as one block (select all statements, then Execute) so that @cid is set and the DELETE affects one row; replace these screenshots with the new output.

P3: audit log after hiring

```
-- Check the audit log and reopen the job
SELECT * FROM Application_Status_Log ORDER BY Log_ID DESC;
```

Figure 94: Audit log query

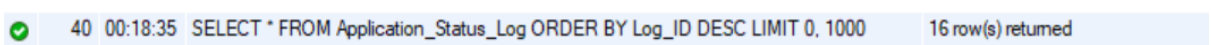


Figure 95: Audit log: output

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

Log_ID	Application_ID	Old_Status	New_Status	Changed_At
16	8	Applied	Rejected	2026-10-26 00:10:20
15	3	Applied	Rejected	2026-10-26 00:10:20
14	1	Interview Scheduled	Hired	2026-10-26 00:10:20
13	2	Shortlisted	Interview Scheduled	2026-10-25 23:58:18
12	6	Applied	Rejected	2026-10-22 23:18:55
11	2	Applied	Shortlisted	2026-10-22 23:18:55
10	4	Applied	Interview Scheduled	2026-10-22 23:18:55
9	1	Applied	Interview Scheduled	2026-10-22 23:18:55
8	8	NULL	Applied	2026-10-22 23:18:55
7	7	NULL	Applied	2026-10-22 23:18:55
6	6	NULL	Applied	2026-10-22 23:18:55
5	5	NULL	Applied	2026-10-22 23:18:55
4	4	NULL	Applied	2026-10-22 23:18:55
3	3	NULL	Applied	2026-10-22 23:18:55
2	2	NULL	Applied	2026-10-22 23:18:55
1	1	NULL	Applied	2026-10-22 23:18:55
NULL	NULL	NULL	NULL	NULL

Application_Status_Log 3

Figure 96: Audit log: the hire, the two automatic rejections and earlier changes

```
UPDATE Job SET Status = 'Open' WHERE Job_ID = 1;
```

Figure 97: Job 1 reopened for the application demo

41 00:20:20 UPDATE Job SET Status = 'Open' WHERE Job_ID = 1 1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

Figure 98: Reopen: output

10. Conclusion

The project delivers a recruitment platform that goes beyond CRUD. Relational data lives in MySQL, where constraints, triggers and transactions guarantee consistency regardless of the client, and unstructured resume and job text lives in a vector database, where semantic similarity adds value that keyword search cannot. The Next.js layer adds authentication, role-based access and a usable interface over the database features.

10.1 What was achieved

- A normalised 12-table schema with keys, CHECK/UNIQUE constraints and referential actions chosen per relationship.
- Six advanced SQL features: transactions, views, function and procedures, triggers, indexing and full-text search, plus vector similarity search.
- A hybrid search design in which Qdrant finds nearest neighbours and MySQL filters and enriches the results.
- Security by design: hashed passwords, JWT in HttpOnly cookies, role checks on every API, parameterised queries and secrets in environment variables.
- Systematic tests showing that invalid data is rejected by the database itself.

10.2 Limitations and future work

- Resumes are entered as text; PDF and DOCX parsing would remove manual copy-paste.
- Ranking could combine cosine similarity with the skill-match percentage into one score.
- Email notifications for status changes and interview invitations.
- Pagination and caching for large datasets; automated end-to-end tests.

10.3 Team contributions and repository

Member	Contribution
Sumedh B Rao	Schema extensions, views, functions, procedures and triggers, important queries, data dictionary; authentication, jobs, applications and interviews APIs and pages
Srujan Yalla	Seed data and test cases; Qdrant design, embedding pipeline and semantic search; dashboard, candidates, profile and match pages; documentation

Source code, SQL scripts and setup instructions are maintained in a Git/GitHub repository, with individual contributions visible in the commit history.

Repository link:

https://github.com/sumedh007rao-prog/DMBS-MINI-PROJECT-